Knowledge Representation

Outlines

- Definition and importance of Knowledge, Issues in Knowledge Representation, Knowledge Representation Systems, Properties of Knowledge Representation Systems,
- Types of Knowledge Representation Systems: Semantic Nets, Frames, Conceptual Dependencies, Scripts, Rule Based Systems (Production System),
- Propositional Logic(PL): Syntax, Semantics, Formal logic-connectives, truth tables, tautology, validity, well-formed-formula, Inference using Resolution, Backward Chaining and Forward Chaining,
- Predicate Logic(FOPL): Syntax, Semantics, Quantification, Inference with FOPL: By converting into PL (existential and universal instantiation), Unification and lifting, Inference using resolution,
- Handling Uncertain Knowledge, Radom Variables, Prior and Posterior Probability, Inference using Full Joint Distribution, Bayes' Rule and its use, Bayesian Networks, Reasoning in Belief Networks,
- Fuzzy Logic: Fuzzy Sets, Membership in Fuzzy Set, Fuzzy Rule base Systems.

What is Knowledge ?

- Knowledge is a theoretical or practical understanding of a subject or a domain. Knowledge is also the sum of what is currently known.
- Knowledge is "information combined with experience, context, interpretation, and reflection. It is a high-value form of information that is ready to apply to decisions and actions." (T. Davenport et al., 1998)
- Knowledge is “human expertise stored in a person’s mind, gained through experience, and interaction with the person‘s environment." (Sunasee and Sewery, 2002)
- Humans are best at understanding, reasoning, and interpreting knowledge. Human knows things, which is knowledge and as per their knowledge they perform various actions in the real world. **But how machines do all these things comes under knowledge representation and reasoning.**
- **Knowledge** in the context of AI refers to the information, facts, concepts, and relationships that an intelligent system uses to make decisions and solve problems.

- Knowledge consists of information that has been:
  - interpreted,
  - categorized,
  - applied, experienced and revised.
- In **AI**, knowledge based agents use a process of **reasoning** over an internal **representation** of knowledge to decide what actions to take.

Importance of Knowledge:

- **Decision Making:** Knowledge allows AI systems to make informed decisions, reason logically, and provide solutions to complex problems.
- **Efficiency:** Properly structured knowledge can significantly enhance the efficiency of problem-solving processes.
- **Learning:** AI systems can learn from existing knowledge to improve their performance over time.
- **Communication:** Knowledge enables systems to communicate and understand human instructions or queries effectively.

- Computer requires a well-defined problem description to process and provide well-defined acceptable solution.
- To collect fragments of knowledge we need first to formulate a description in our spoken language and then represent it in formal language so that computer can understand.
- The computer can then use an algorithm to compute an answer. This process is illustrated as,

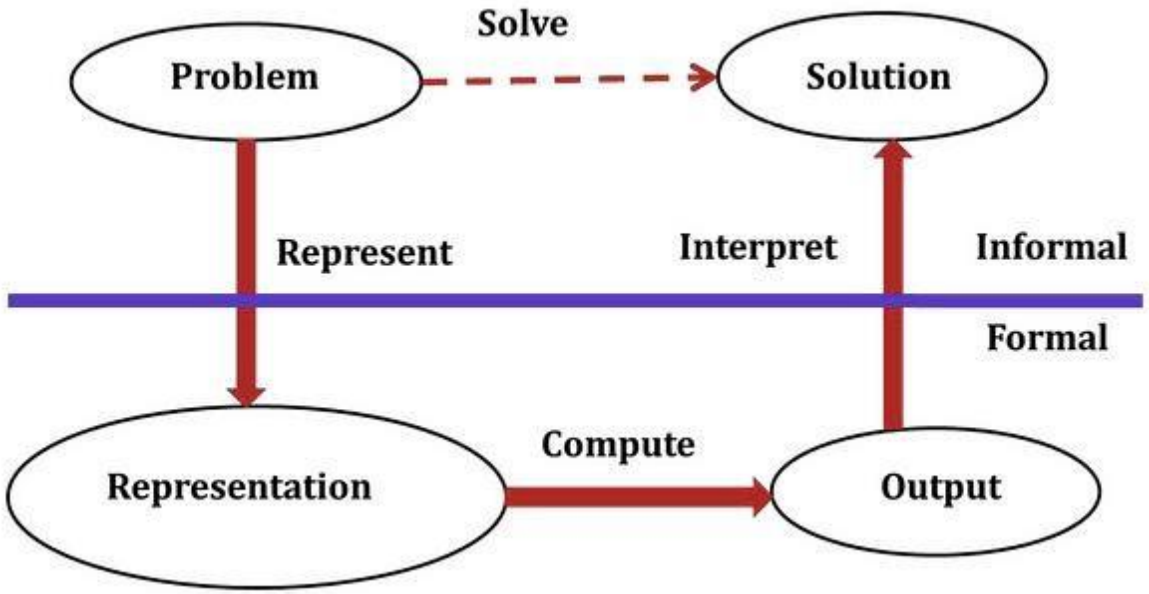


Figure: Knowledge Representation Framework.

The steps are:

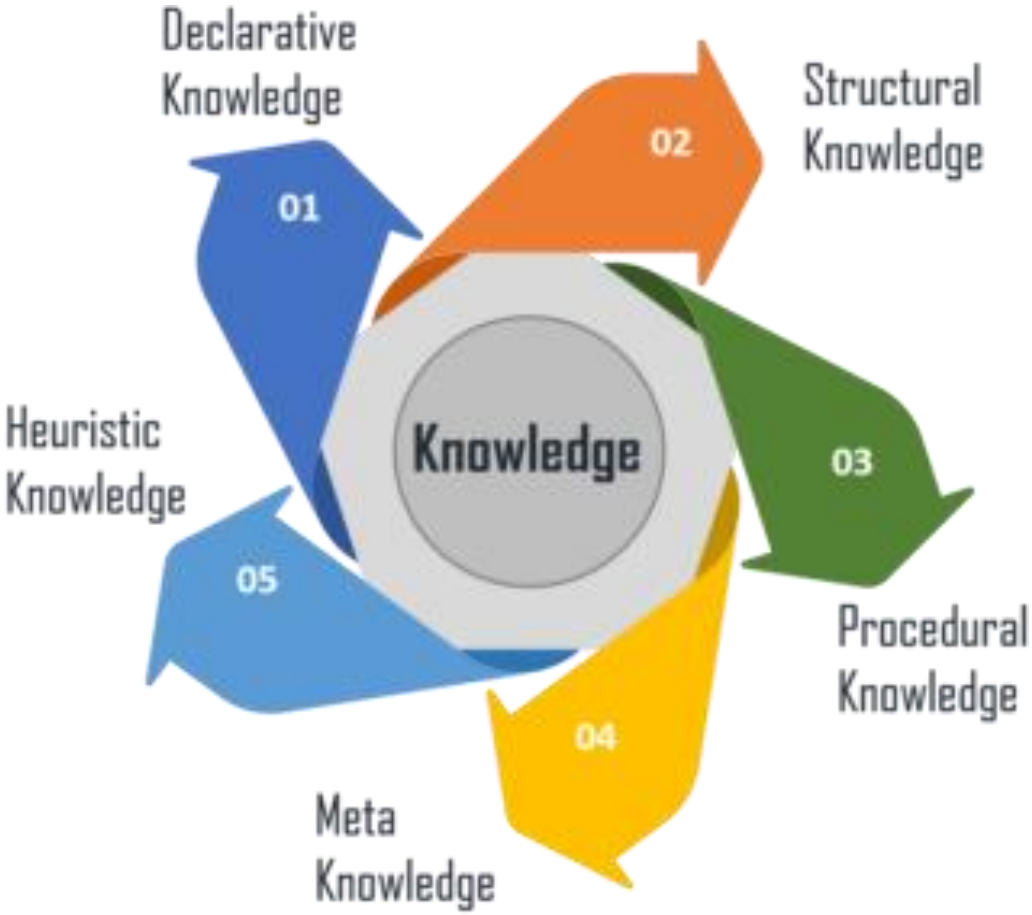
- The informal formalism of the problem takes place first.
- It is then represented formally and the computer produces an output.
- This output can then be represented in an informally described solution that user understands or checks for consistency.

The Problem solving requires,

- Formal knowledge representation, and
- Conversion of informal knowledge to formal knowledge that is conversion of implicit knowledge to explicit knowledge.

- Following are the kind of knowledge which needs to be represented in AI systems:
  - **Object:** All the facts about objects in our world domain. e.g., "A car has wheels, an engine, and seats"
  - **Events:** Events are the actions which occur in our world.
  - **Performance:** It describe behavior which involves knowledge about how to do things.
  - **Meta-knowledge:** It is knowledge about what we know.
  - **Facts:** Facts are the truths about the real world and what we represent. (e.g., “Kathmandu is the capital of Nepal"
  - **Knowledge-Base:** The central component of the knowledge-based agents is the knowledge base. It is represented as KB. The Knowledgebase is a group of the Sentences (Here, sentences are used as a technical term and not identical with the English language).

Different Types of Knowledge



- **Declarative Knowledge**: It includes concepts, facts, and objects and expressed in a declarative sentence. Thus it deals with the description of something.

Examples:

  - "The capital of Nepal is Kathmandu.“
  - "Water boils at 100°C.“
  - "Elephants are the largest land animals."

- **Structural Knowledge**: It is a basic problem-solving knowledge that describes the relationship between concepts and objects.

Examples:

  - Understanding that a "tree" is a plant, and a "plant" is a living organism.
  - Recognizing the hierarchical relationship in a company's organizational structure.

- **Procedural Knowledge** – This is responsible for knowing how to do something and includes rules, strategies, procedures, etc. This type of knowledge highly depends on the task we are trying to accomplish.

Examples:

  - Knowing how to ride a bicycle.
  - Steps involved in solving a quadratic equation.
  - The procedure for baking a cake.

- **Meta Knowledge** – Meta Knowledge defines knowledge about other types of Knowledge.

Examples:

  - Knowing how to organize a knowledge database.
  - Understanding which learning strategies are most effective for different types of content.
  - Awareness of different problem-solving approaches and when to use them.

- **Heuristic Knowledge** – This represents some expert knowledge in the field or subject.

Examples:

- A doctor using clinical heuristics to diagnose diseases based on symptoms.
- A chess player using heuristics to decide on the best moves.
- A software developer applying design patterns to solve common programming problems.

Knowledge Representation (KR) System and It’s properties

- **Knowledge representation (KR)** is the study of how knowledge about the world can be represented and what kinds of reasoning can be done with that knowledge.
- Knowledge Representation is the method used to encode knowledge in Intelligent Systems.
- Knowledge-based agents are artificial intelligence systems that use knowledge to perform their tasks. For example, they make deductions, decisions, and conclusions based on available knowledge.
- Knowledge-based agents are essential in several fields, including medicine, engineering, and finance, where they are used to solve complex problems.
- Knowledge-based agents are composed of two main parts:
  - **Knowledge-base and**
  - **Inference system**.

- **Knowledge-Base:** The knowledge base of a knowledge-based agent is a collection of knowledge that the agent uses to make decisions.
- The knowledge can be explicit, such as rules or facts, or implicit, such as relationships or patterns. The knowledge base is stored in a database or a knowledge representation system.
- **Inference System:** The inference system of a knowledge-based agent is responsible for using the knowledge in the knowledge base to make decisions.
- The inference system uses reasoning methods, such as deduction, induction, and abduction, to infer new knowledge from existing knowledge.
- Generating new facts by the inference system allows the agent to update its knowledge base. The inference system operates primarily through two rules, which are referred to as forward chaining and backward chaining.

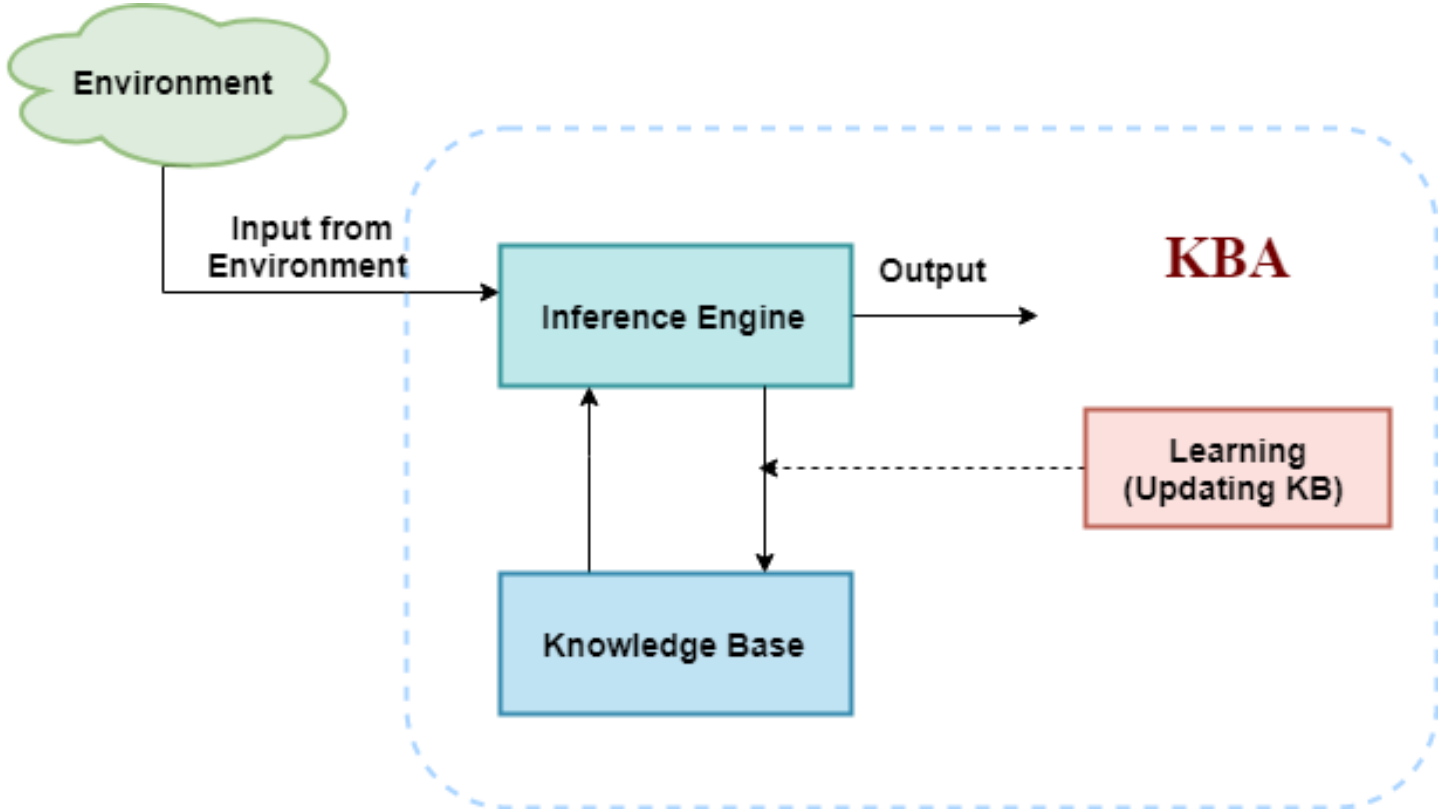


Figure: The architecture of knowledge-based agent.

Requirements of Knowledge Representation System

- A good knowledge representation system must possess the following properties.

1. **Representational Accuracy:** KR system should have the ability to represent all kind of required knowledge.
2. **Inferential Adequacy:** KR system should have ability to manipulate the representational structures to produce new knowledge corresponding to existing structure.
3. **Inferential Efficiency:** The ability to direct the inferential knowledge mechanism into the most productive directions by storing appropriate guides.
4. **Acquisitional efficiency:** The ability to acquire the new knowledge easily using automatic methods.

Issues in Knowledge Representation

- **The fundamental goal of knowledge Representation is to facilitate inference (conclusions) from knowledge.**
- The issues that arise while using KR techniques are many. Some of these are explained below.
  - **Important attributes**
  - **Relationship between attributes**
  - **Choosing Granularity**
  - **Set of objects**
  - **Finding the Right structures**

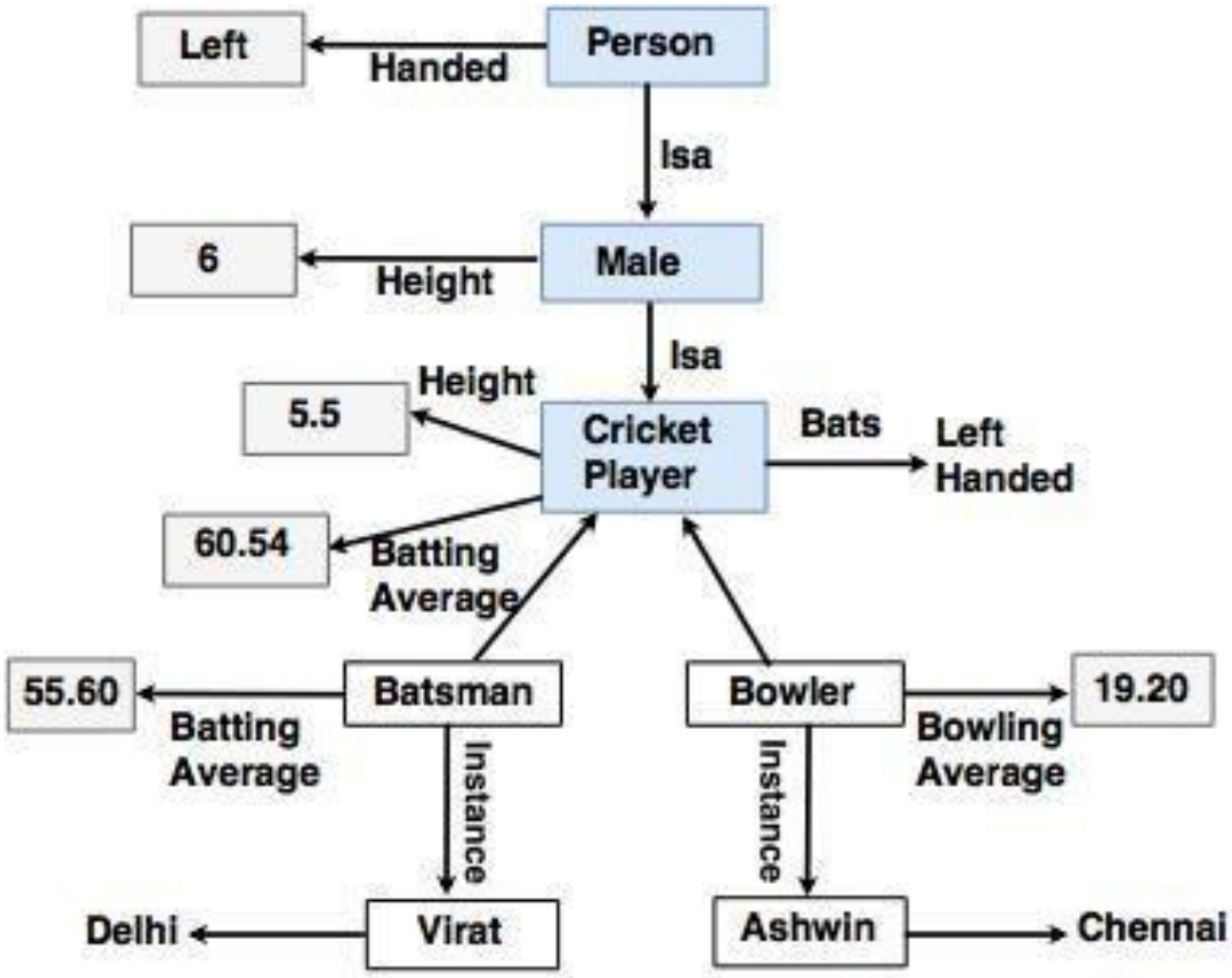


Fig: Inheratable Knowledge Representation

Important attributes:

Is there a property of an object so fundamental that it appears in nearly every area of concern?

- Two general importance terms, "instance" and "is a," are credited. The fact that these qualities facilitate property inheritance makes them significant.

Relationships among attributes:

- Basically, the attributes used to describe objects are nothing but the entities.
- The relationship between attributes of an object, is independent of specific knowledge they encode, and may hold properties like,
  - Inverse: The entities are related to each other in many different ways.
  - Existence in a is a hierarchy: generalization-specialization relationships are important for attributes because they support inheritance.

- Technique for reasoning about values: Several kinds of information are used in reasoning, like, height: must be in a unit of length, Age: of a person cannot be greater than the age of person's parents. The values are often specified when a knowledge base is created.
- Single value attributes: This is about a specific attribute that is guaranteed to take a unique value. For example, a baseball player can at time have only a single height and be a member of only one team.

Representing sets of objects:

- How should sets of objects be represented?
- Here are some properties of objects which satisfy the condition of a set together but not as individual.

Examples: A person is very strict at a company but he is very calm at his home.

Consider the assertion made in the sentences:

"There are more sheep than people in Australia", and "English speakers can be found all over the world."

These facts can be described by including an assertion to the sets representing people, sheep, and English.

Choosing the granularity of representation:

- While deciding the granularity of representation, it is necessary to know the following:
  a. What are the primitives and at what level should the knowledge be represented?
  b. What should be the number (small or large) of low-level primitives or high-level facts?
  c. High-level facts may be insufficient to draw the conclusion while Low-level primitives may require a lot of storage.

Examples: Depending on the level of audience we should represent knowledge.

Suppose that we are interested in following facts:

John spotted Alex.

Now, this could be represented as "Spotted (agent(John), object (Alex))”

Such a representation can make it easy to answer questions such as: Who spotted Alex?

Finding the right structure as needed:

- Given a large amount of knowledge stored in a database, how can relevant parts are accessed when they are needed?
- This is about access to right structure for describing a particular situation.
- This requires, selecting an initial structure and then revising the choice.
- While doing so, it is necessary to solve following problems:
  - How to perform an initial selection of the most appropriate structure.
  - How to fill in appropriate details from the current situations.
  - How to find a better structure if the one chosen initially turns out not to be appropriate.
  - What to do if none of the available structures is appropriate.
  - When to create and remember a new structure.

There is no good, general purpose method for solving all these problems. Some knowledge representation techniques solve some of these issues.

Types of Knowledge Representation Systems

- Knowledge can be represented in different ways. The structuring of knowledge and how designers might view it, as well as the type of structures used internally are considered.
- Different knowledge representation techniques are
  - **Logical Representation**
  - **Semantic Network**
  - **Frame**
  - **Conceptual Graphs**
  - **Conceptual Dependency**
  - **Script**
  - **Production System**

Logical Representation:

- A logic is a formal language with some **definite rules** which deal with propositions and has no ambiguity in representation.
- It consists of precisely defined syntax and semantics which supports the sound inference. Each sentence can be translated into logics using **syntax and semantics**.
- The logic may be different types like **propositional logic, predicate logic, temporal logic, description logic** etc.

Syntax	Semantics
• It decides how we can construct legal sentences in logic. • It determines which symbol we can use in knowledge representation. • Also, how to write those symbols.	• Semantics are the rules by which we can interpret the sentence in the logic. • It assigns a meaning to each sentence.

- Syntax (the structure of sentences) and propositional logic its semantics (the way in which the truth of sentences is determined).

Advantages:

- Logical representation helps to perform logical reasoning.
- This representation is the basis for the programming languages.

Disadvantages:

- Logical representations have some restrictions and are challenging to work with.
- This technique may not be very natural, and inference may not be very efficient.

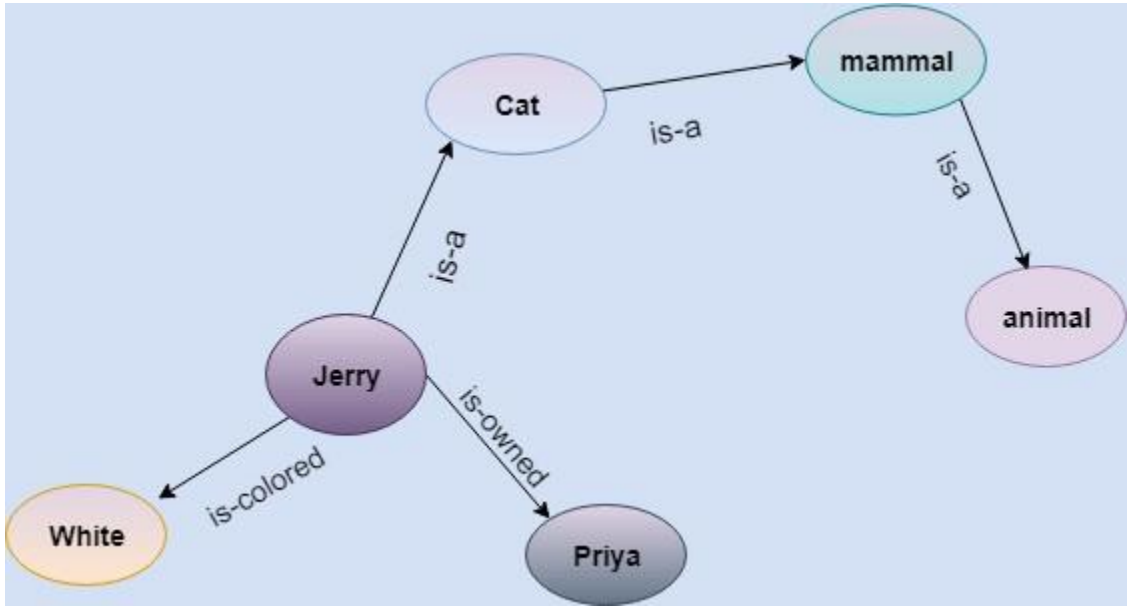
Semantic Network:

- A semantic network is a graphical knowledge representation technique, representation system is primarily on network structure.
- The main idea behind semantic net is that the meaning of a concept comes, from the ways in which it is connected to other concepts.
- The semantic network consists of different **nodes** and **arcs**.
- Each node should contain the information about objects and each arc should contain the relationship between objects.
- Semantic nets are used to find relationships among objects by spreading activation about from each of two nodes and seeing where the activation met this process is called **intersection search**.
- This representation consist of mainly two types of relations:
    - **IS-A relation (Inheritance)**
    - **Kind-of-relation**

Example: Following are some statements which we need to represent in the form of nodes and arcs.

Statements:

- Jerry is a cat.
- Jerry is a mammal
- Jerry is owned by Priya.
- Jerry is brown colored.
- All Mammals are animal.



Advantages:

- Semantic networks are a natural representation of knowledge.
- Also, it conveys meaning in a transparent manner.
- These networks are simple and easy to understand.

Disadvantages:

- Semantic networks take more computational time at runtime.
- Also, these are inadequate as they do not have any equivalent quantifiers.
- These networks are not intelligent and depend on the creator of the system.

Frame:

- A frame is a collection of attributes and associated values that describe some entity in the world. Frames are general record like structures which consist of a collection of **slots** and **slot values**. The slots may be of any size and type.
- Slots typically have names and values or subfields called **facets**. Facets may also have names and any number of values. A frame may have any number of slots, a slot may have any number of facets, each with any number of values.
- A slot contains information such as attribute value pairs, default values, condition for filling a slot, pointers to other related frames and procedures that are activated when needed for different purposes.
- Sometimes a frame describes an entity in some absolute sense, sometimes it represents the entity from a particular point of view. A single frame taken alone is rarely useful.
- A frame is also known as **slot-filter knowledge representation** in artificial intelligence.

Syntax of a frame

```
(<frame name>
(<slot1> (<facet1> <value 1>.........<value n1>)
        (<facet2> <value1>............<value n2>)
        .
        .
        .
        .
        .
        (<facet n> <value1>............ <value nn>))
(<slot 2> (<facet1> <value 1>......<value n1>)
        (<facet2><value2>............<value n2>)
        .
        .
        ))
```

Example 1: frame for a book

Slots **(Attributes)**	Filters **(Values)**
Title	Artificial Intelligence
Genre	Computer Science
Author	Peter Norvig
Edition	Third Edition
Year	1996
Page	1152

Example 2: Create a frame of the person Ram who is an Engineer. He is of 25. His wife name is Sita. They have two children Muna and Gita. They live in 100 kps street in the city of ktm in Nepal. The zip code is 123456.

Soln:

(Ram

(PROFESSION (VALUE Engineer))

(AGE (VALUE 25))

(WIFE (VALUE Sita))

(CHILDREN (VALUE Muna, Gita))

(ADDRESS

(STREET (VALUE 100 kps))

(CITY(VALUE ktm))

(COUNTRY(VALUE Nepal))

(ZIP (VALUE 123456))))

Conceptual Graphs:

- As the name indicates, it tries to capture the concepts about the events and represents them in the form of a graph. A concept may be individual or generic.
- An individual concept has a type field followed by a reference field.
- For example, person : Ram. Here person indicates type and Ram indicates reference.
- An individual concept should be represented within a rectangle in graphical representation and within a square bracket in linear representation.
- The generic concept should be represented within an oval in graphical representation and within a parenthesis in linear representation. Conceptual graph is a basic building block for associative network.
- Concepts like AGENT, OBJECT, INSTRUMENT, PART are obtained from a collection of standard concepts.

Example: “Ram is eating an apple”

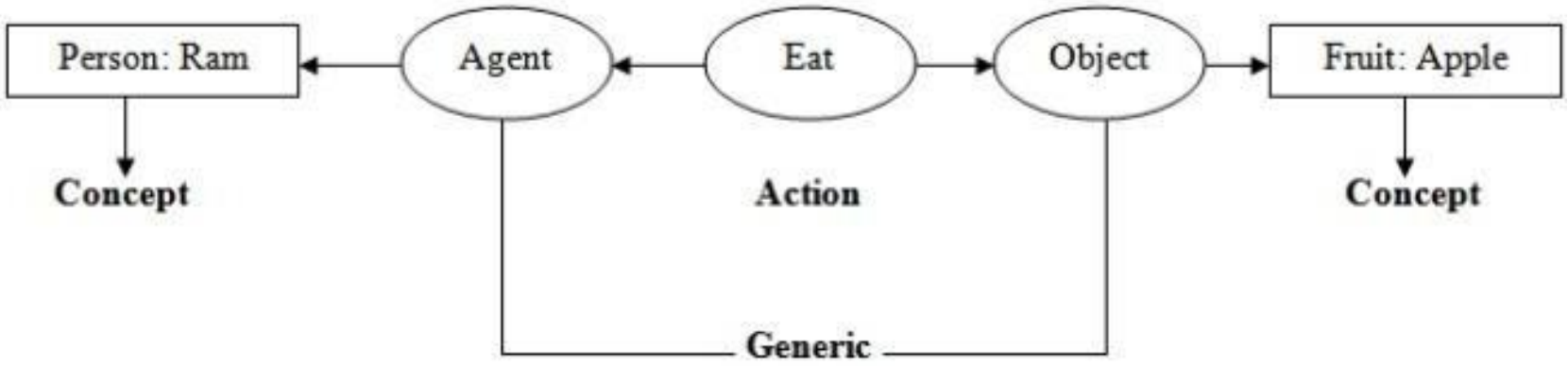


Figure Graphical Representation

[Person: Ram]←——(Agent)←—— [Eat]——→(Object)——→[Fruit: Apple]

Linear Representation

Conceptual Dependency (CD):

- Conceptual Dependency originally developed to represent knowledge acquired from natural language input.
- It is based on the use of a limited number of primitive concepts and rules of formation to represent any natural language statement.
- Conceptual dependency theory is based on the use of knowledge representation methodology was primarily developed to understand and represent natural language structures.
- The conceptual dependency structures were originally developed by Roger C SChank in 1977.
- The conceptual dependency representation captures maximum concepts to provide canonical form of meaning of sentences.

- The goals of this theory are:
  - To help in the drawing of inference from sentences.
  - To be independent of the words used in the original input.
- Generally there are six primitives from which the conceptual dependency structure can be described.

a. ACTs : Actions

b. PPs : Objects (Picture Producers)

c. AAs : Modifiers of Actions (Action Aiders)

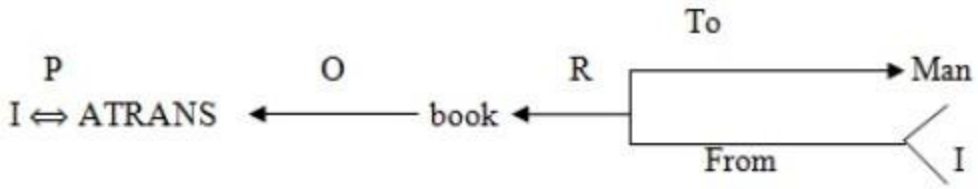
d. PAs : Modifiers of PPs (Picture Aiders)

e. TS : Time of action

f. LOC : Locations

- Conceptual dependency provides both a structure and a specific set of primitives at a particular level of granularity, out of which representation of particular pieces of information can be constructed.

For example: I gave a book to the man. CD representation is as follow:



Where ←: Direction of dependency

Double arrow indicates two way link between actor and action.

P: Past Tense, ATRANS: One of the primitive acts used by the theory, I: Instrument (The means or tool used to perform the action), O: The objective case relation, R: Recipient case Relation, D: destination e.g. going home.

- It should be noted that this representation is the same for different sayings with the same meaning.

For example

- I gave the man a book
- The man got a book form me
- The book was given to man by me etc.

In CD, representation of actions are built from a set of primitive acts.

1) **ATRANS:** Transfer of an abstract relationship (give, accept, take)
2) **PTRANS:** Transfer the physical location of an object (Go, Come, Run, Walk)
3) **MTRANS:** Transfer the mental information (Tell)
4) **PROPEL:** Application of physical force to an object (push, pull, throw)
5) **MOVE:** Movement of a body part by its owner (kick).
6) **GRASP:** Grasping of an object by an action (clutch)
7) **INGEST:** Ingestion of an object by an animal (eat)
8) **EXPEL:** Expel from an animal body (cry)
9) **MBUILD:** Building new information out of old (decide)
10) **SPEAK:** Production of sounds (say)

Following conceptual tenses are used in CD representation.

1) O : Object case relationship
2) R : Recipient case relationship
3) P : Past
4) F : Future
5) Nil : Present
6) T : Transition
7) Ts : Start Transition
8) Tf : Finisher Transition
9) K : Continuing
10) ? : Interrogative
11) / : Negative
12) C : Conditional

Rule 1: PP $\Longleftrightarrow$ ACT

It describes the relationship between an actor and an event, he/she causes.

E.g. Ram ran

Ram $\overset{P}{\Longleftrightarrow}$ PTRANS

Where P: Past Tense

Rule 2: PP $\Longleftrightarrow$ PA

It describes the relationship between a PP and PA where the PA indicates one characteristics of PP. E.g. Ram is tall

Ram $\overset{Nil}{\Longleftrightarrow}$ Tall or Ram $\overset{Nil}{\Longleftrightarrow}$ Height (> Average)

Rule 3: PP $\Longleftrightarrow$ PP

It describes the relationship between two PPs where one PP is defined by other.

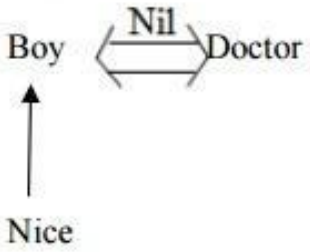
E.g. Ram is a doctor

Ram $\overset{Nil}{\Longleftrightarrow}$ Doctor

Rule 4: PP $\uparrow$ PA or PA $\downarrow$ PP

It describes the relationship between the PP and PA, where PA indicates one attributes of PP.

E.g. A nice boy is a doctor



Rule 5: PP

It describes the relationship between 3 PP's where one PP is the owner of another PP.

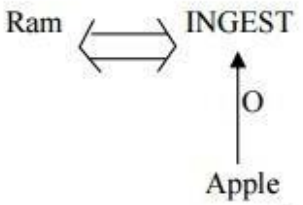
E.g. Ram's Cat

Cat

Ram

Rule 6: Act $\xleftarrow{O}$ PP Where O: Object

It describes the relationship between the PP and ACT. Where PP indicates the object of that action. E.g. Ram is eating an apple.

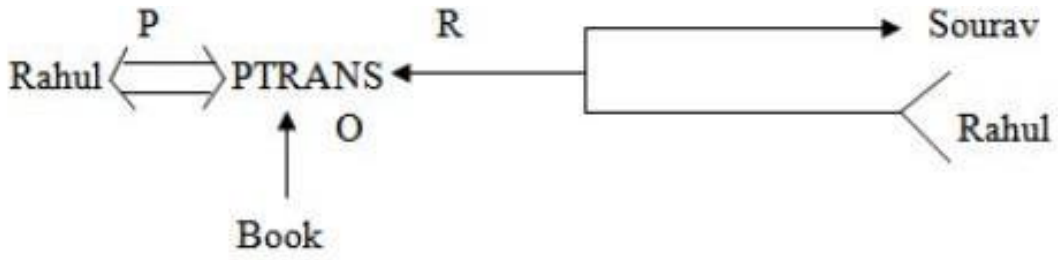


Rule 7: ACT

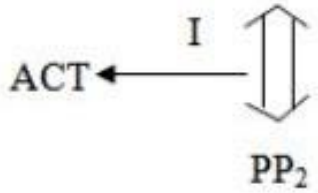
(R: Recipient)

Here one PP describes the recipient and another PP describes the donner

E.g. Rahul gave a book to sourav.



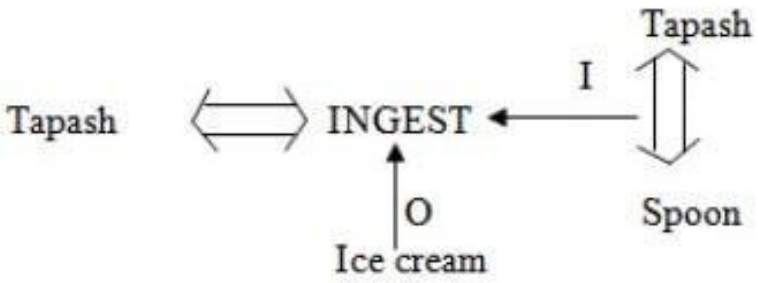
Rule 8:



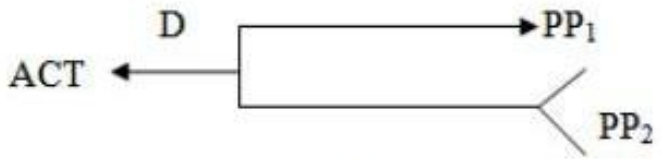
(I: Instrument used in the action)

Here PP_1 indicates the agent and PP_2 indicates the object that is used in the action.

E.g. Tapash ate the ice cream with the spoon.

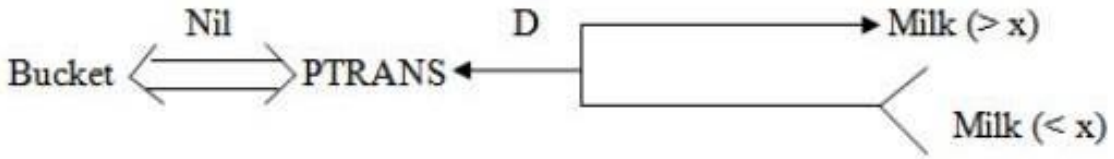


Rule 9:



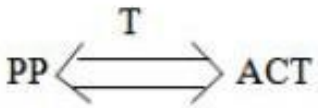
Here D indicates destination, PP_1 indicates destination and PP_2 indicates the source.

E.g. the bucket is filled with milk.



xindicates the average milk and the source i.e. bucket is dry which is hidden.

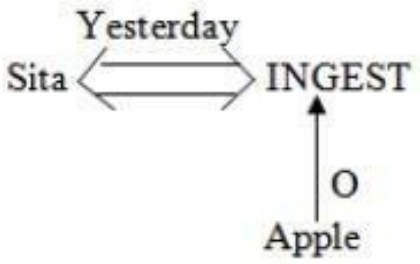
Rule 10:



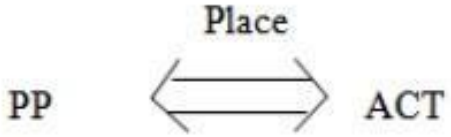
(T: Time)

It describes the relationship between a conceptualization and the time at which the event is described occurs.

E.g. Sita ate the apple yesterday.

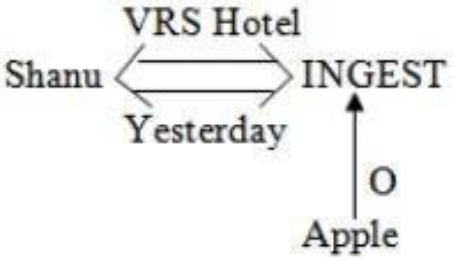


Rule 11:

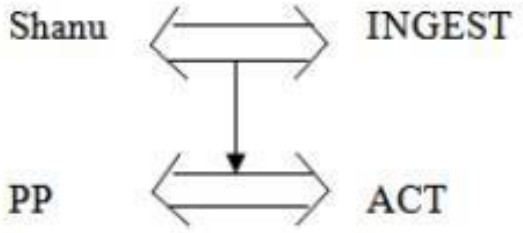


It describes the relationship between a conceptualization and the place at which it is occurred.

E.g. Shanu ate the apple at VRS hotel yesterday

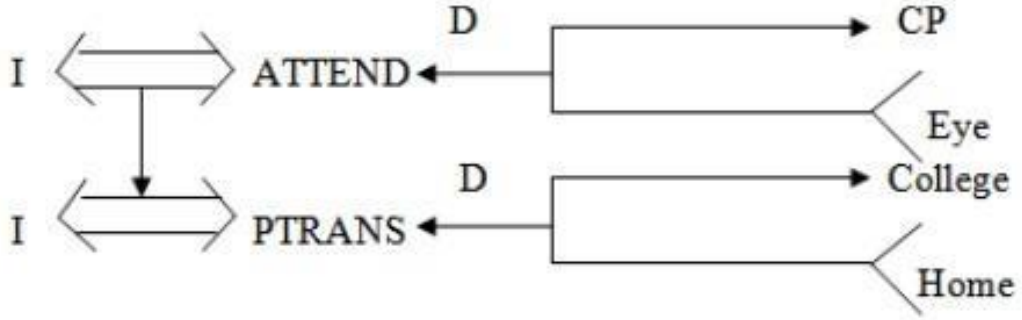


Rule 12:



It describes the relationship between one conceptualization with another.

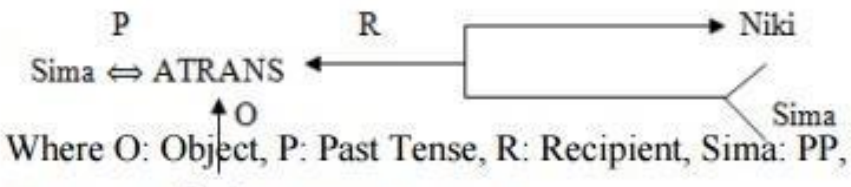
E.g. while I was going to college, I saw a snake



(Where CP: Conscious Processor i.e. the combination of all sense organs like eye, ear, nose etc.)

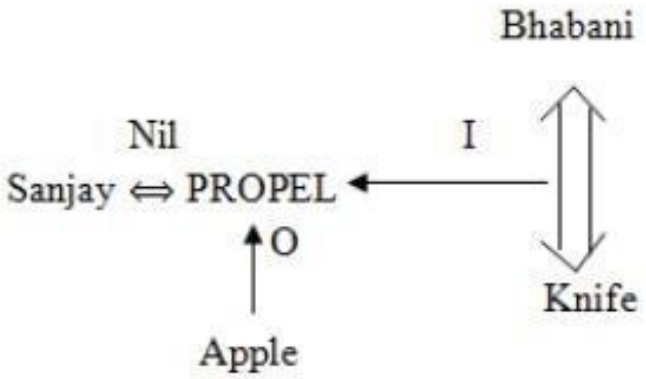
By using the above rules we can represent any sentence. Let us visualize few examples on conceptual dependency.

1) Sima gave a book to Niki

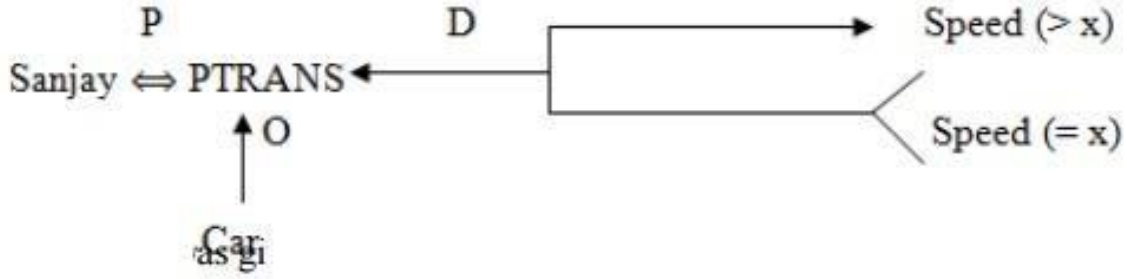


Where O: Object, P: Past Tense, R: Recipient, Sima: PP, Book: PP, Niki: PP, ATRANS: give

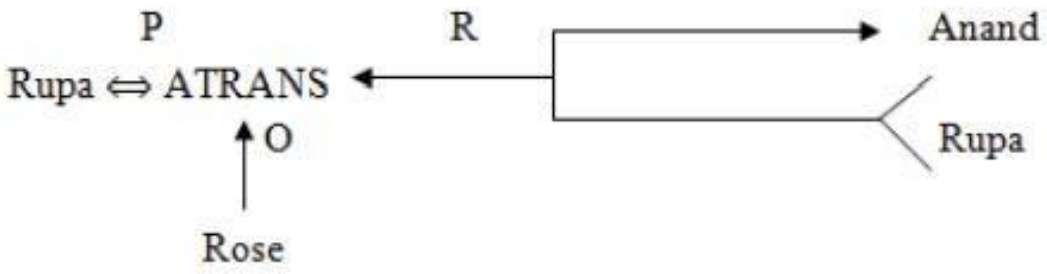
2) Bhabani cuts an apple with a knife



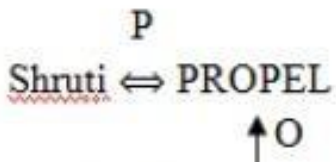
3) Sanjay drove the car fast



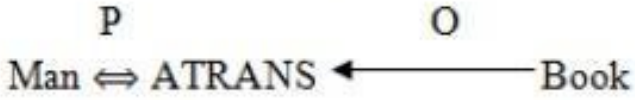
4) The rose was given by Rupa to Anand



5) Shruti pushed the door.



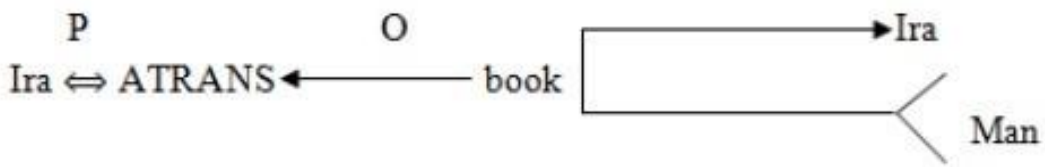
6) The man took a book



Here man is the doctor and book is the object of the action took.

7) My grandfather told me a story

8) Ira gave the man a dictionary



Script:

- Scripts are frame like structures used to represent commonly occurring experiences such as going to restaurant, visiting a doctor.
- A script is a structure that describes a stereotyped sequence of events in a particular context.
- A script consist of a set of slots. Associated with each slot may be some information about what kinds of values it may contain as well as a default value to be used if no other information is available.
- Scripts are useful because in the real world, there are no patterns to the occurrence of events. These patterns arise because of clausal relationships between events. The events described in a script form a giant casual chain.
- The beginning of the chain is the set of entry conditions which enable the first events of the script to occur. The end of the chain is the set of results which may enable later events to occur. The headers of a script can all serve as indicators that the script should be activated.
- A script has the ability to predict events that has not explicitly been observed.

- An important use of scripts is to provide a way of building a single coherent interpretation from a collection of observation.
- Scripts are less general structures than are frames and so are not suitable for representing all kinds of knowledge.

Advantages

- Ability to predict events.
- A single coherent interpretation maybe builds up from a collection of observations.

Disadvantages

- Less general than frames.
- May not be suitable to represent all kinds of knowledge

A script has various components like:

1. **Entry conditions**: It must be true before the events described in the script can occur. E.g. in a restaurant script the entry condition must be the customer should be hungry and the customer has money.
2. **Tracks:** It specifies particular position of the script. E.g. In a supermarket script the tracks may be cloth gallery, cosmetics gallery etc.
3. **Props:** Objects involved in the script. Tickets, seats etc. are other objects that the person deals with while watching a match.
4. **Roles:** It specifies the various stages of the script. E.g. In a restaurant script the scenes may be entering, ordering etc.
5. **Scenes:** The sequence of events that occur.
6. **Results:** Condition that will be true after events in script occurred. E.g. In a restaurant script the result must be true if the customer is pleased. The customer has less money.

Example 1: Now let us look on a movie script description according to the above component.

a. **Script name :** Movie

b. **Track :** CINEMA HALL

c. **Roles :** Customer(c), Ticket seller(TS), Ticket Checker(TC), Snacks Sellers (SS)

d. **Props :** Ticket, snacks, chair, money, Ticket, chart

e. **Entry conditions :** The customer has money, The customer has interest to watch movie.

f. **Scenes:** (various scenes)

Scene 1: Entering into the cinema hall

- C PTRANS C into the cinema hall
- C ATTEND eyes towards the ticket counter
- C PTRANS C towards the ticket counters
- C ATTEND eyes to the ticket chart

- C MBUILD to take which class ticket
- C MTRANS TS for ticket
- C ATRANS money to TS
- TS ATRANS ticket to C

Scene-2 : Entering into the main ticket check gate

- C PTRANS C into the queue of the gate
- C ATRANS ticket to TC
- TC ATTEND eyes onto the ticket
- TC MBUILD to give permission to C for entering into the hall
- TC ATRANS ticket to C
- C PTRANS C into the picture hall.

Scene-3 :Entering into the picture hall

- C ATTEND eyes into the chair
- TC SPEAK where to sit
- C PTRANS C towards the sitting position
- C ATTEND eyes onto the screen

Scene-4: Ordering snacks

- C MTRANS SS for snacks
- SS ATRANS snacks to C
- C ATRANS money to SS
- C INGEST snacks

Scene-5: Exiting

- C ATTEND eyes onto the screen till the end of picture
- C MBUILD when to go out of the hall
- C PTRANS C out of the hall

g. Result:

- The customer is happy
- The customer has less money

Example 2: A script for watching a cricket match

Script : watching a match

Track: Cricket match

Props:

- Tickets
- Seat
- Match

Roles:

- Person (who wants to Watch a match) – P
- Booking Clerk – BC
- Security personal – SP
- Ticket Checker – TC

Entry Conditions:

- P wants to watch match
- P has a money

Results:

- P saw a match
- P has less money
- P is happy (if his team has won) or not (if his team has lost) or some other problem at stadium

Various Scene

Scene 1: Going to a stadium

- P PTRANS P to the stadium
- P ATTEND eyes to ticket counter

Scene 2: Buying ticket

- P PTRANS P to ticket counter
- P MTRANS ticket requirement to BC
- P MTRANS stand information to BC
- BC ATRANS ticket to P

Scene 3: Going inside stadium and sitting on a seat

- P PTRANS P into Stadium
- SPMOVE security check P
- TC ATTEND eyes on ticket POSS by P
- TC MOVE to tear part of ticket
- TC MTRANS (showed seat) to P
- P PTRANS P to seat
- P MOVES P to sitting position

Scene 4: Watching a match

- P ATTEND eyes on match
- P MBUILD (moments) from the match

Scene 5: Exiting

- P PTRANS P out of Stadium

Rule-Based Systems (Production System):

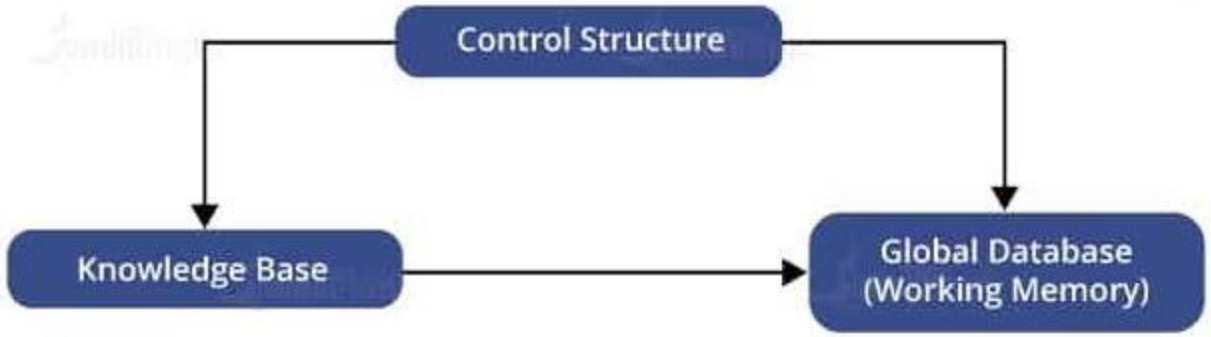
- Also known as Knowledge-Based System.
- A system that applies human-made rules to store, sort and manipulate data.
- Rule based system in AI require a set of facts or source of data, and a set of rules for manipulating that data.
- ‘IF X happens THEN do Y’.

Examples:

- IF (at bus stop AND bus arrives) THEN action (get into the bus)
- IF (on the bus AND paid AND empty seat) THEN action (sit down).
- IF (on bus AND unpaid) THEN action (pay charges).
- IF (bus arrives at destination) THEN action (get down from the bus).

Components of a Production System

- The architecture of a production system in Artificial Intelligence consists of production rules, a database, and the control system.



Knowledge Base

- It holds the domain knowledge that is necessary for problem solving.
- In a rules-based system, the knowledge gets represented as a set of rules.
- Every rule specifies a relation, recommendation, directive, strategy or heuristic and has the IF (condition) THEN (action) structure. If a condition is passed through a production rule and gets satisfied by the global database, then the rule is successfully applied.

Global Database

- A global database used as a central data structure. A database contains all the necessary data and information required for the successful completion of a task.
- It can be divided into two parts as permanent and temporary.
- The permanent part of the database consists of fixed actions, whereas the temporary part alters according to circumstances.

Control System

- The control system checks the applicability of a rule.
- It helps decide which rule should be applied and terminates the process when the system gives the correct output.
- It also resolves the conflict of multiple conditions arriving at the same time. The strategy of the control system specifies the sequence of rules that compares the condition from the global database to reach the correct result.

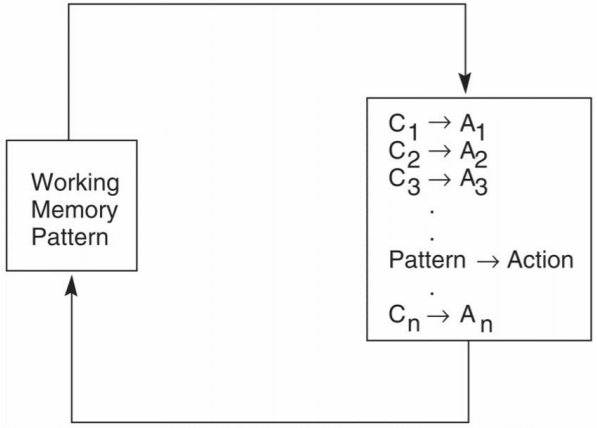
Example: Sorting a String in a Production System

Initial String (Short term memory): ‘cbaca’

Final String: ‘aabcc’

Mechanism for sorting a string using the production system in AI as,

- The production rules that we use for sorting will be enabled when it satisfies the condition by finding the sub-string in memory.
- When a particular rule is selected, it replaces the matched string by the string present on the right-hand side of the production rule.
- The loop of production rules will iterate until it finds the correct output.



Let us look at a basic production rule that can be used in this case:

1. ba -> ab
2. ca -> ac
3. cb -> bc

Now, the below diagram will show the execution of the rules for converting the string.

Iteration #	Working memory	Conflict set	Rule fired
0	cbaca	1, 2, 3	1
1	cabca	2	2
2	acbca	2, 3	2
3	acbac	1, 3	1
4	acabc	2	2
5	aacbc	3	3
6	aabcc	Ø	Halt

Advantages:

- The production rules are expressed in natural language.
- The production rules are highly modular and can be easily removed or modified.

Disadvantages:

- It does not exhibit any learning capabilities and does not store the result of the problem for future uses.
- During the execution of the program, many rules may be active. Thus, rule-based production systems are inefficient.

Propositional Logic (PL)

Logic: If the rules of the world are presented formally, then a decision maker can use logical reasoning to make rational decisions.

- Several types of logic:
  - Propositional Logic
  - First-Order Logic
  - Fuzzy Logic
  - Bayesian Logic
  - Modal Logic etc.

A logic includes:

- **Syntax**: what is a correctly-formed sentence?
- **Semantics**: what is the meaning of a sentence?
- **Inference procedure** (reasoning, entailment): what sentence logically follows given knowledge?

- A **proposition** is a declarative statement which is either true or false but not both.
- If a proposition is true, then we say it has a **truth value** of "**true**"; if a proposition is false, its **truth value** is "**false**".

Examples:

a) "Man is Mortal", it returns truth value —True

b) The Sun rises from West, it returns truth value —False

c) 3+3= 7, it returns truth value —False

- The following sentences are not Proposition:
  a) "A is less than 2". It is because unless we give a specific value of A, we cannot say whether the statement is true or false.
  b) "Close the door", and "Is it hot outside ?"are not propositions.

Propositional Logic Syntax:

- **Atomic sentences** consist of a **single proposition symbol**. Each such symbol stands for a proposition that can be true or false.
- We can use any symbol for a representing a proposition, such A, B, C, P, Q, R, ..etc.

Examples:

a) 2+2 is 4, it is an atomic proposition as it is a **true** fact.

b) "The Sun is cold" is also a proposition as it is a **false** fact.

- **Complex sentences** are constructed from simpler sentences, using parentheses and operators called **logical connectives.**

Examples:

a) "It is raining today, and street is wet."

b) "Ankit is a doctor, and his clinic is in Mumbai."

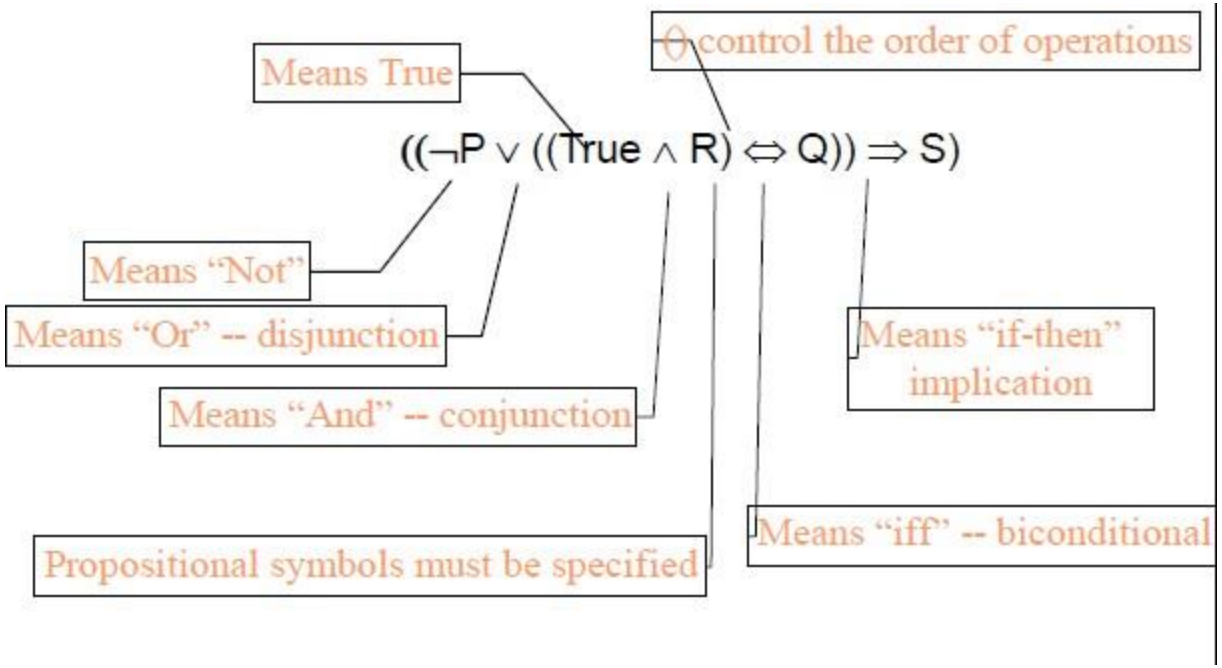
- There are five connectives in common use:
  - ¬ (not). A sentence such as ¬W1,3 is called the **negation** of W1,3. A **literal** is either an atomic sentence (**a positive literal**) or a negated atomic sentence (a **negative literal**).
  - ∧ (and). A sentence whose main connective is ∧, such asW1,3 ∧ P3,1, is called a **conjunction**; its parts are the conjuncts. (The ∧ looks like an “A” for “And.”)
  - ∨ (or). A sentence whose main connective is ∨, such as (W1,3 ∧ P3,1) ∨ W2,2, is a **disjunction**; its parts are disjuncts—in this example, (W1,3 ∧ P3,1) andW2,2.
  - ⇒ (implies). A sentence such as (W1,3 ∧ P3,1) ⇒ ¬W2,2 is called an implication (or conditional). Its premise or antecedent is (W1,3 ∧ P3,1), and its **conclusion** or **consequent** is ¬W2,2. Implications are also known as rules or **if–then** statements.
  - ⇔ (if and only if). The sentence W1,3 ⇔ ¬W2,2 is a **biconditional.**

$$
\begin{array}{rcl}
Sentence & \rightarrow & AtomicSentence \mid ComplexSentence \\
AtomicSentence & \rightarrow & True \mid False \mid P \mid Q \mid R \mid \ldots \\
ComplexSentence & \rightarrow & (\ Sentence\ ) \\
 & \mid & \neg\ Sentence \\
 & \mid & Sentence \wedge Sentence \\
 & \mid & Sentence \vee Sentence \\
 & \mid & Sentence \Rightarrow Sentence \\
 & \mid & Sentence \Leftrightarrow Sentence \\
\text{OPERATOR PRECEDENCE} & : & \neg, \wedge, \vee, \Rightarrow, \Leftrightarrow
\end{array}
$$

Figure: A BNF (Backus–Naur Form) grammar of sentences in propositional logic, along with operator precedences, from highest to lowest

Example: $\neg P \vee Q \wedge R \Rightarrow S$ is equivalent to $((\neg P) \vee (Q \wedge R)) \Rightarrow S$

- **A sentence (well-formed formula) is defined as follows**
  - A symbol is a sentence
  - If S is a sentence, then $\neg S$ is a sentence
  - If S is a sentence, then (S) is a sentence
  - If S and T are sentences, then $(S \vee T)$, $(S \wedge T)$, $(S \rightarrow T)$, and $(S \leftrightarrow T)$ are sentences
  - A sentence results from a finite number of applications of the above rules



Examples of PL sentences

- P means "It is hot“
- Q means "It is humid“
- R means "It is raining“
- P ^ Q => R

"If it is hot and humid, then it is raining"

- Q => P

"If it is humid, then it is hot"

- Q

"It is humid."

Semantics:

- The semantics defines the rules for determining the truth of a sentence with respect to a particular model. In propositional logic, a model simply sets the truth value —true or false—for every proposition symbol.
- For example, if the sentences in the knowledge base make use of the proposition symbols P, Q, and R, then one possible model is m1 = {P=false, Q=false, R=true}. With three proposition symbols, there are 2^3 =8 possible models.
- All sentences are constructed from atomic sentences and the five connectives.(below figure)

How to compute the truth of atomic sentences?

- The truth value of Atomic sentences can be determined by using the following rules :
  - True is true in every model and False is false in every model.
  - The truth value of every other proposition symbol must be specified directly in the model.
  - For example, in the model m1 given earlier, P is false.

How to compute the truth of sentences formed with each of the five connectives?

- For complex sentences, we have five rules, which hold for any subsentences P and Q (atomic or complex) in any model m (here "iff" means "if and only if"):
  - $\neg$ P is true iff P is false in m.
  - P∧Q is true iff both P and Q are true in m.
  - P∨Q is true iff either P or Q is true in m.
  - P⇒Q is true unless P is true and Q is false in m.
  - P⇔Q is true iff P and Q are both true or both false in m.
- The rules can also be expressed with **truth tables** that specify the truth value of a complex sentence for each possible assignment of truth values to its components. Truth tables for the five connectives are given in Figure below.

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	*false*	*true*	*false*	*false*	*true*	*true*
false	*true*	*true*	*false*	*true*	*true*	*false*
true	*false*	*false*	*false*	*true*	*false*	*false*
true	*true*	*false*	*true*	*true*	*true*	*true*

Figure : Truth tables for the five logical connectives.

- From these tables, the truth value of any sentence s can be computed with respect to any model m by a simple recursive evaluation.

Properties

- Validity(Tautology)
- Satisfiability (contingency)
- Un-Satisfiability (Contradictory)
- Equivalent
- Entailment
- Completeness
- Soundness

Validity:

- A sentence is valid if it is true in all models,
  e.g., *True*, $A \vee \neg A$, $A \Rightarrow A$, $(A \wedge (A \Rightarrow B)) \Rightarrow B$

- Valid sentences are also known as tautologies. Every valid sentence is logically equivalent to True.
- **Example :** Prove $[(A \rightarrow B) \wedge A] \rightarrow B$ is a tautology
- Solution: The truth table is as follows,

A	B	$A \rightarrow B$	$(A \rightarrow B) \wedge A$	$[(A \rightarrow B) \wedge A] \rightarrow B$
True	True	True	True	True
True	False	False	False	True
False	True	True	False	True
False	False	True	False	True

As we can see every value of $[(A \rightarrow B) \wedge A] \rightarrow B$ is "True", it is a tautology.

Satisfiability:

- A sentence is satisfiable if it is true in, or satisfied by, some model. E.g., A ∨ A
- Satisfiable sentences are also known as **Contingency.**
- **Example:** Prove (A∨ B)∧ (¬A)a contingency
- Solution: The truth table is as follows

A	B	A ∨ B	¬ A	(A ∨ B) ∧ (¬ A)
True	True	True	False	False
True	False	True	False	False
False	True	True	True	True
False	False	False	True	False

As we can see every value of (A∨B)∧(¬A) has both “True” and “False”, it is a contingency.

Un-Satisfiability (Contradictory) :

- A sentence is un-satisfiable if it is true in no models. E.g., $A \wedge \neg A$
- Un-Satisfiable sentences are also known as contradictory sentences.
- Example: Prove $(A \vee B) \wedge [(\neg A) \wedge (\neg B)]$ is a contradiction

Solution: The truth table is as follows :

A	B	$A \vee B$	$\neg A$	$\neg B$	$(\neg A) \wedge (\neg B)$	$(A \vee B) \wedge [(\neg A) \wedge (\neg B)]$
True	True	True	False	False	False	False
True	False	True	False	True	False	False
False	True	True	True	False	False	False
False	False	False	True	True	True	False

As we can see every value of $(A \vee B) \wedge [(\neg A) \wedge (\neg B)]$ is ―False‖, it is a contradiction.

Propositional Equivalences:

- Two statements X and Y are logically equivalent if any of the following two conditions hold :
  - The truth tables of each statement have the same truth values.
  - The bi-conditional statement $X \Leftrightarrow Y$ is a tautology.
- Example : Prove $\neg(A \vee B)$ and $[(\neg A) \wedge (\neg B)]$ are equivalent Testing by 1st method (Matching truth table)

A	B	$A \vee B$	$\neg (A \vee B)$	$\neg A$	$\neg B$	$[(\neg A) \wedge (\neg B)]$
True	True	True	False	False	False	False
True	False	True	False	False	True	False
False	True	True	False	True	False	False
False	False	False	True	True	True	True

Here, we can see the truth values of $\neg(A \vee B)$ and $[(\neg A) \wedge (\neg B)]$ are same, hence the statements are equivalent.

Testing by 2nd method (Bi-conditionality)

A	B	$\neg(A \vee B)$	$[(\neg A) \wedge (\neg B)]$	$[\neg(A \vee B)] \Leftrightarrow [(\neg A) \wedge (\neg B)]$
True	True	False	False	True
True	False	False	False	True
False	True	False	False	True
False	False	True	True	True

As $[\neg(A \vee B)] \Leftrightarrow [(\neg A) \wedge (\neg B)]$ is a tautology, the statements are equivalent.

Standard logical equivalences:

- Some standard logical equivalences in propositional logic is as shown below:
- The symbols α, β, and γ stand for arbitrary sentences of propositional logic.

$$
\begin{aligned}
(\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\
(\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\
((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\
((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\
\neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\beta \Rightarrow \neg\alpha) && \text{contraposition} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\alpha \vee \beta) && \text{implication elimination} \\
(\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) && \text{biconditional elimination} \\
\neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{De Morgan} \\
\neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{De Morgan} \\
(\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\
(\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge
\end{aligned}
$$

Entailment:

- Entailment means that one thing follows from another:
  - E.g ., x + y =4 entails 4=x + y
  - KB as a haystack and α as a needle. Entailment is like the needle being in the haystack; inference is like finding it.
- Entailment is a relationship between sentences that is based on semantics
- Knowledge base KB entails sentence α if and only if α is true in all worlds where KB is true . : KB |= α
- If an inference algorithm i can derive α from KB, we write:

$$KB \vdash_i \alpha$$

- which is pronounced ―α is derived from KB by i‖ or ―i derives α from KB.‖

Soundness:

- An inference algorithm that derives only entailed sentences is called sound or truth preserving. Soundness is a highly desirable property.
- An unsound inference procedure essentially makes things up as it goes along—it announces the discovery of nonexistent needles.

Completeness:

- The property of completeness is also desirable: an inference algorithm is complete if it can derive any sentence that is entailed.
- For real haystacks, which are finite in extent, it seems obvious that a systematic examination can always decide whether the needle is in the haystack.
- For many knowledge bases, however, the haystack of consequences is infinite, and completeness becomes an important issue.
- Fortunately, there are complete inference procedures for logics that are sufficiently expressive to handle many knowledge bases.

Inference Methods

- The process by which a conclusion is drawn from given premises.
- But logic is concerned with: does the truth of the conclusion follow from that of the premises?
- Several basic methods for determining whether a given set of premises propositionally entails a given conclusion.
  - Truth Table Method (Enumeration Method)
  - Deductive (Proof) Systems
  - Resolution

Truth-table approach:

A two steps procedure;

- Generate table for all possible interpretations.
- Check whether the sentence evaluates to true whenever *KB* evaluates to true.
- Example: $\alpha = A \vee B$ and $KB = (A \vee C) \wedge (B \vee \neg C)$

A	B	C	$A \vee C$	$(B \vee \neg C)$	KB	α
True	*True*	*True*	*True*	*True*	*True*	*True*
True	*True*	*False*	*True*	*True*	*True*	*True*
True	*False*	*True*	*True*	*False*	*False*	*True*
True	*False*	*False*	*True*	*True*	*True*	*True*
False	*True*	*True*	*True*	*True*	*True*	*True*
False	*True*	*False*	*False*	*True*	*False*	*True*
False	*False*	*True*	*True*	*False*	*False*	*False*
False	*False*	*False*	*False*	*True*	*False*	*False*

KB entails α

Deductive (Proof) Systems:

- Done by applying rules of inference directly to the sentences in our knowledge base to construct a proof of the desired sentence without consulting models.
- Example:
    - All men are mortal.
    - Ram is a man.
    - Therefore, Ram is mortal.
- If the number of models is large but the length of the proof is short, then Deductive proof can be more efficient than model checking.

Rules for Propositional Logic

Inference Rules:

- Logically equivalences are used as a inference rules and additional inference rules are:

(MP) Modes Ponens (Implication-elimination):

$$\frac{\alpha \Rightarrow \beta, \quad \alpha}{\beta}$$

- The notation means that, whenever any sentences of the form α ⇒ β and α are given, then the sentence β can be inferred. i.e. If both sentences in the premise are true then conclusion is true.
- For example, if (WumpusAhead ∧WumpusAlive) ⇒ Shoot and (WumpusAhead ∧WumpusAlive) are given, then Shoot can be inferred.

(AE)And-Elimination: from a conjunction, any of the conjuncts can be inferred.

$$\frac{\alpha \wedge \beta}{\alpha}$$

- For example, from (WumpusAhead ∧WumpusAlive), WumpusAlive can be inferred.

(NE) Negation-elimination: $\frac{\neg\neg A}{A}$

- All inference rules are **sound.**

Normal Forms

- Sentences in the propositional logic can be transformed into one of the normal forms. This can simplify the inferences.
- There are two major normal forms of statements in propositional logic. They are :

Conjunctive Normal Form (CNF):

- Conjunction of clauses (clauses include disjunctions of literals)
- E.g., $(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$

Disjunctive Normal Form (DNF):

- Disjunction of terms (terms include conjunction of literals)
- E.g., $(A \wedge \neg B) \vee (B \wedge \neg C) \vee (\neg C \wedge D \wedge A)$

Conversion to CNF

- A sentence that is expressed as a conjunction of disjunctions of literals is said to be in conjunctive normal form (CNF). A sentence in CNF that contains only k literals per clause is said to be in k-CNF.

Algorithm:

- Eliminate $\leftrightarrow$ rewriting $P \leftrightarrow Q$ as $(P \rightarrow Q) \wedge (Q \rightarrow P)$
- Eliminate $\rightarrow$ rewriting $P \rightarrow Q$ as $\neg P \vee Q$
- Use De Morgan laws to push $\neg$ inwards:
  - rewrite $\neg(P \wedge Q)$ as $\neg P \vee \neg Q$
  - rewrite $\neg(P \vee Q)$ as $\neg P \wedge \neg Q$
- Eliminate double negations: rewrite $\neg\neg P$ *as* P

- Use the distributive laws to get CNF:
  - rewrite $(P \wedge Q) \vee R$ as $(P \vee R) \wedge (Q \vee R)$
- Flatten nested clauses:
  - $(P \wedge Q) \wedge R$ as $P \wedge Q \wedge R$
  - $(P \vee Q) \vee R$ as $P \vee Q \vee R$

Example: conversion to CNF

$B \Leftrightarrow (A \vee C)$

- Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow ß$ with $(\alpha \Rightarrow ß) \wedge (ß \Rightarrow \alpha)$.
  - $(B \Rightarrow (A \vee C)) \wedge ((A \vee C) \Rightarrow B)$

- Eliminate $\Rightarrow$, replacing $\alpha \Rightarrow ß$ with $\neg \alpha \vee ß$.
  - $(\neg B \vee A \vee C) \wedge (\neg(A \vee C) \vee B)$

- Move $\neg$ inwards using de Morgan's rules and double-negation:
  - $(\neg B \vee A \vee C) \wedge ((\neg A \wedge \neg C) \vee B)$

- Apply distributivity law ($\wedge$ over $\vee$) and flatten:
  - $(\neg B \vee A \vee C) \wedge (\neg A \vee B) \wedge (\neg C \vee B)$

Proof by Resolution

Resolution Rules:

(UR) Unit Resolution: Unit resolution rule takes a clause (a disjunction of literals) and a literal and produces a new clause. Single literal is also called unit clause.

$$\frac{\ell_1 \vee \cdots \vee \ell_k, \qquad m}{\ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k}$$

For example:

$$\frac{\alpha \vee \beta, \quad \neg\beta}{\alpha}$$

Where li and m are complementary literals.

Generalized Resolution: The unit resolution rule can be generalized to the full resolution rule,

$$\frac{\ell_1 \vee \cdots \vee \ell_k, \qquad m_1 \vee \cdots \vee m_n}{\ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n}$$

where ℓ_i and m_j are complementary literals.

For example:

$$\frac{\alpha \vee \beta, \quad \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

- Resolution is used with knowledge bases in CNF (or clausal form), and is complete for propositional logic.
- Resolution takes two clauses and produces a new clause containing all the literals of the two original clauses except the two complementary literals.
- In addition the resulting clause should contain only one copy of each literal. The removal of multiple copies of literals is called **factoring.**
- For example, if we resolve $(A \vee B)$ with $(A \vee \neg B)$, we obtain $(A \vee A)$, which is reduced to just A.

Resolution Algorithm:

– Convert KB into CNF

– Add negation of sentence to be entailed into KB i.e. (KB ∧ ¬ α)

– Then apply resolution rule to resulting clauses.

– The process continues until:

– There are no new clauses that can be added

Hence *KB* **does not** entail α

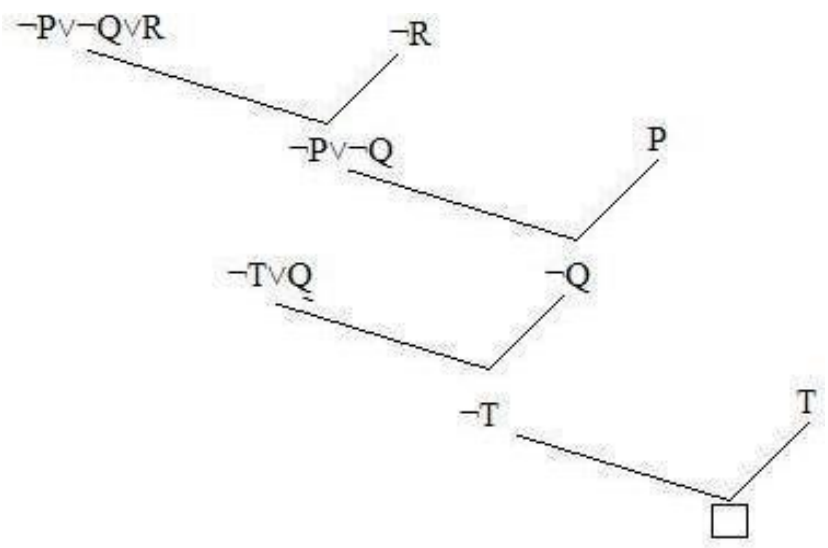
– Two clauses resolve to entail the empty clause.

Hence *KB* **does** entail α

Q. Suppose we are given the axioms shown in the 2nd column of Table below and we want to prove R.

Solution: First we convert the axioms to clause which is already in clause form.

Sl. No	Given Axioms	Converted to Clause Form
1	P	P
2	$(P \wedge Q) \rightarrow R$	$\neg P \vee Q \vee R$
3	$(S \vee T) \rightarrow Q$	$\neg S \vee Q$
4		$\neg T \vee Q$
5	T	T



Then we begin selecting pairs of clauses to resolve together.

Example: Consider the knowledge base given as: $KB = (B \Leftrightarrow (A \vee C)) \wedge \neg B$

Prove that $\neg A$ can be inferred from above KB by using resolution.

<u>Solution:</u>

At first, convert KB into CNF

$$B \Rightarrow (A \vee C)) \wedge ((A \vee C) \Rightarrow B) \wedge \neg B$$

$$(\neg B \vee A \vee C) \wedge (\neg(A \vee C) \vee B) \wedge \neg B$$

$$(\neg B \vee A \vee C) \wedge ((\neg A \wedge \neg C) \vee B) \wedge \neg B$$

$$(\neg B \vee A \vee C) \wedge (\neg A \vee B) \wedge (\neg C \vee B) \wedge \neg B$$

Add negation of sentence to be inferred from KB into KB

Now KB contains following sentences all in CNF

$(\neg B \vee A \vee C)$

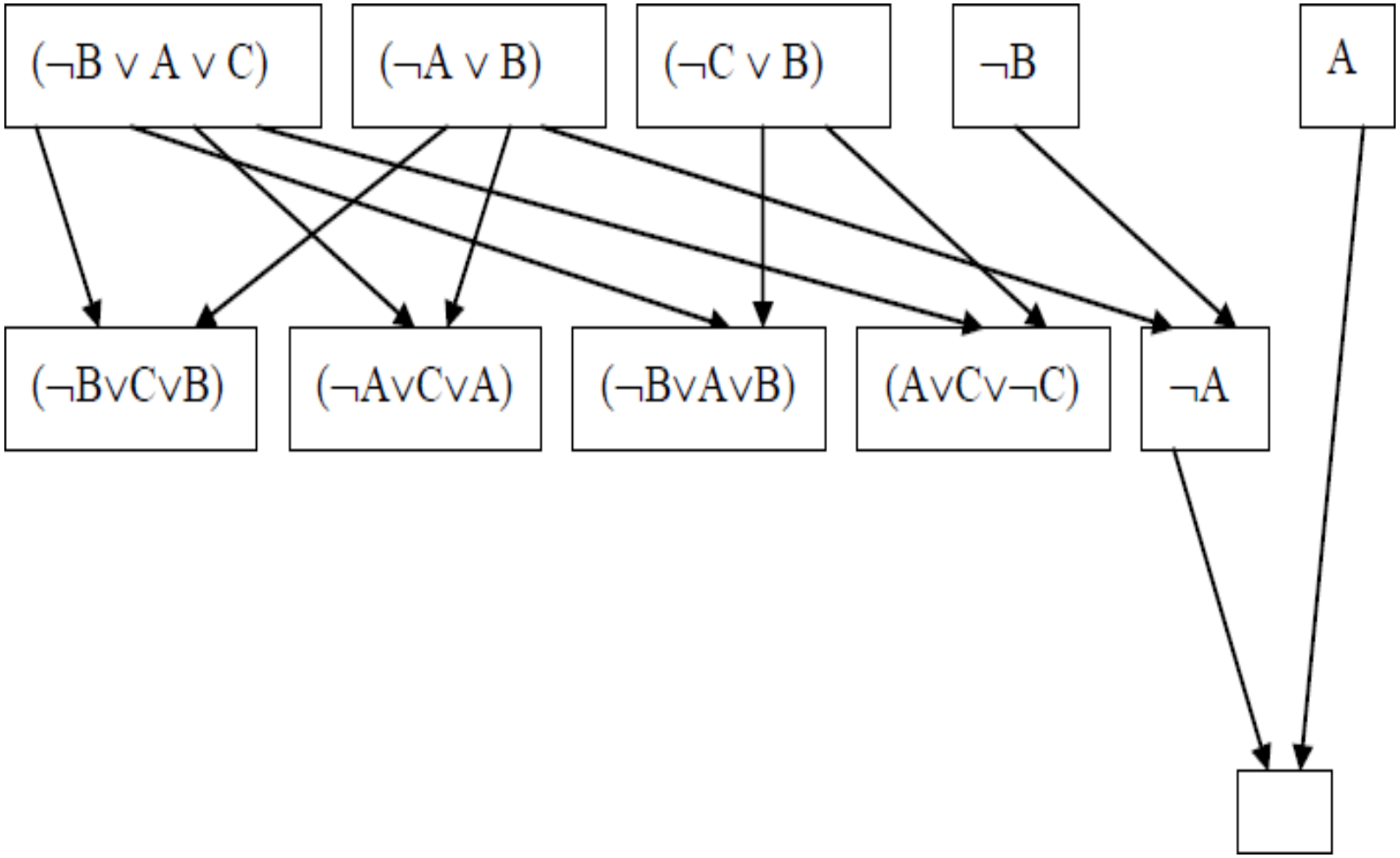
$(\neg A \vee B)$

$(\neg C \vee B)$

$\neg B$

A (negation of conclusion to be proved)

Now use Resolution algorithm


(¬B ∨ A ∨ C)
(¬A ∨ B)
(¬C ∨ B)
¬B
A
(¬B∨C∨B)
(¬A∨C∨A)
(¬B∨A∨B)
(A∨C∨¬C)
¬A

Forward and Backward Chaining

- The completeness of resolution makes it a very important inference model. But in many practical situations full power of resolution is not needed. Real-world knowledge bases often contain only clauses of restricted kind called **Horn Clause.**
- **A Horn clauses is disjunction of literals with at most one positive literal.**
- Example: ($\neg$ p V $\neg$ q V k). It has only one positive literal k. It is equivalent to $p \wedge q \rightarrow k$.
- Three important properties of Horn clause are:
  - ✓ Can be written as an implication.
  - ✓ Inference through forward chaining and backward chaining.
  - ✓ Deciding entailment can be done in a time linear size of the knowledge base.

Forward chaining (FC):

- **(**Forward chaining is sound and complete for Horn KB).

Idea:

– fire any rule whose premises are satisfied in the *KB*,

– add its conclusion to the *KB*, until query is found

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

$B \wedge L \Rightarrow M$

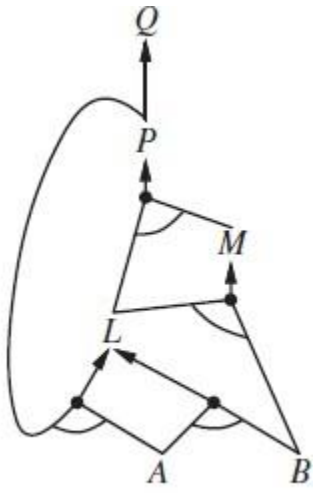
$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L$

A

B

(a)



(b)

Prove that Q can be inferred from above KB

Figure : (a) A set of Horn clauses. (b) The corresponding AND–OR graph.

Solution:

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

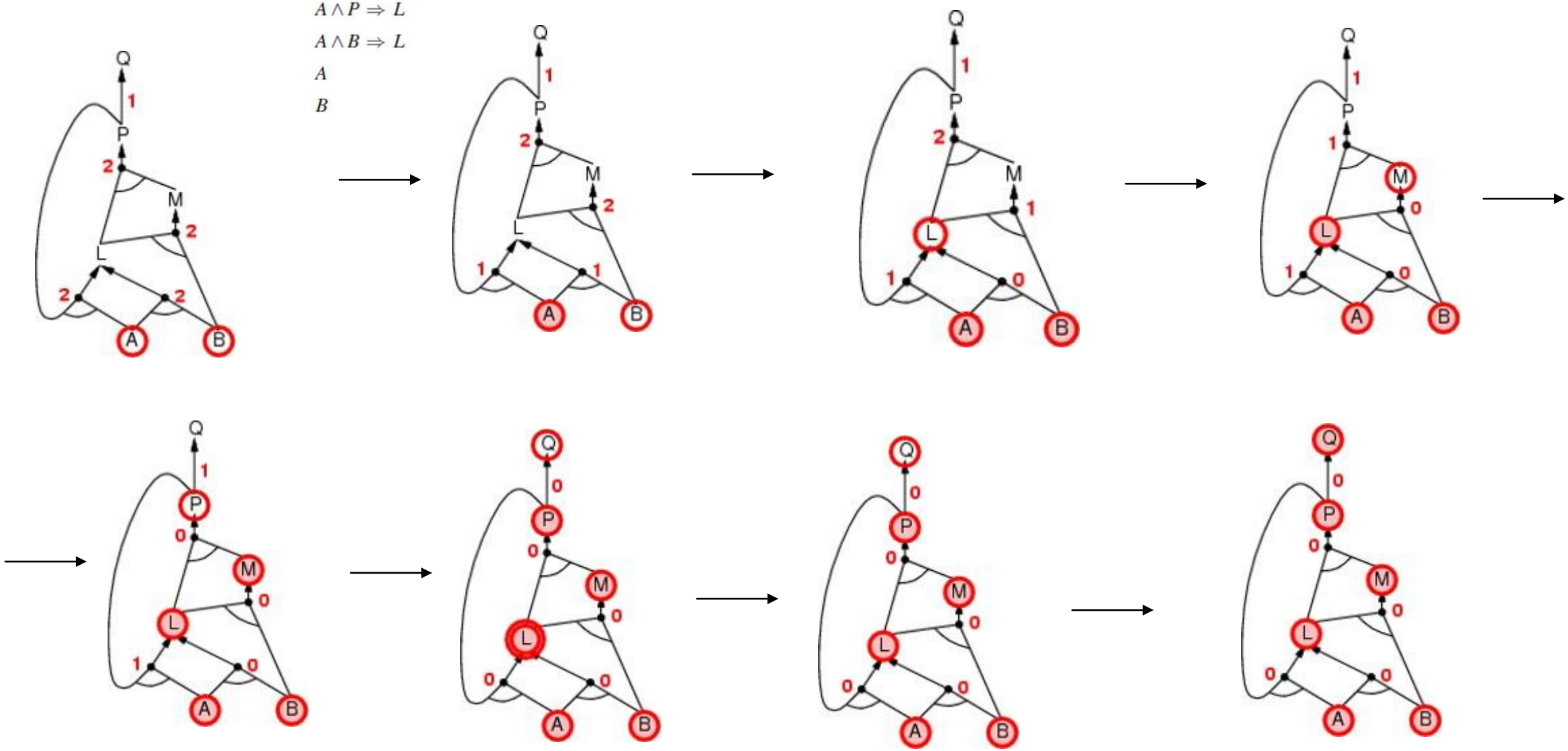
$B \wedge L \Rightarrow M$

$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L$

A

B



Backward chaining (BC):

Idea:

work backwards from the query q: to prove q by BC,

check if q is known already, or

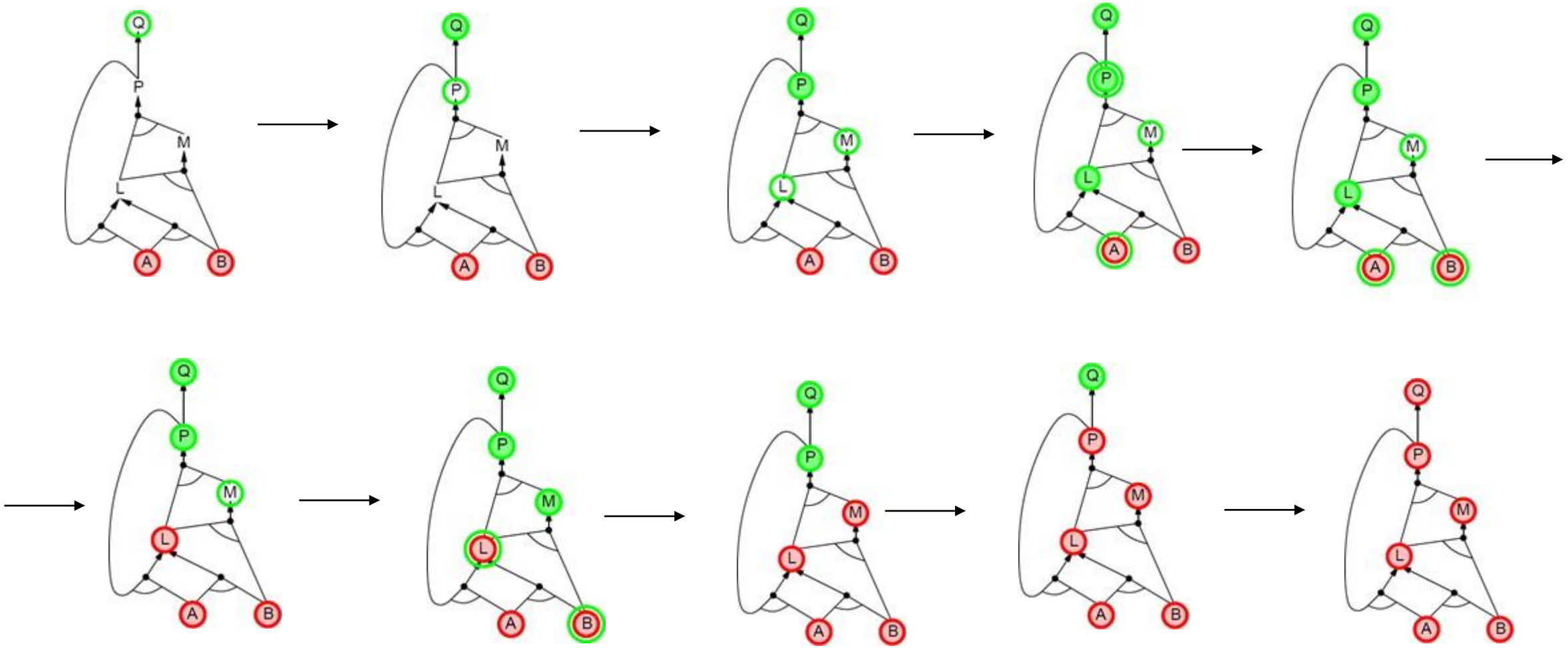
prove by BC all premises of some rule concluding q

For example, for above KB (as in forward chaining above)

$$P \Rightarrow Q$$
$$L \wedge M \Rightarrow P$$
$$B \wedge L \Rightarrow M$$
$$A \wedge P \Rightarrow L$$
$$A \wedge B \Rightarrow L$$
$$A$$
$$B$$

Prove that Q can be inferred from above KB.

Solution:



Forward vs. backward chaining

- FC is **data-driven**, automatic, unconscious processing,

  -e.g., object recognition, routine decisions

- May do lots of work that is irrelevant to the goal
- BC is **goal-driven**, appropriate for problem-solving,

  -e.g., Where are my keys? How do I get into a PhD program?

- Complexity of BC can be much less than linear in size of KB.

Predicate Logic/ First-Order Logic(FOL)

- FOL is another knowledge representation in AI which is an extended part of PL. FOL articulates the natural language statements briefly.

Pros and cons of propositional logic:

- Propositional logic is declarative
- Propositional logic allows partial/disjunctive/negated information

  -(unlike most data structures and databases)
- Propositional logic is compositional:

  -meaning of B ∧ P is derived from meaning of B and of P
- Meaning in propositional logic is context-independent

  -(unlike natural language, where meaning depends on context)
- Propositional logic has very limited expressive power

  -(unlike natural language)

- Propositional logic assumes the world contains facts, where as First-order logic (FOL) models the world contains:
  - **Objects,** which are things with individual identities, Eg., lectures, students, people, houses, numbers, cricket games, companies, cars ...
  - **Properties ,** of objects that distinguish them from others, Eg., blue, oval, even, large, ...
  - **Relations,** that hold among sets of objects, Eg., Brother, bigger-than, outside, part-of, has-color, occurs-after, owns, visits, precedes, ...
  - **Functions,** which are a subset of relations where there is only one ―value‖ for any given ―input‖, Eg., father-of, best-friend, second-half, one more-than, plus,...

Key differences between PL and FOL

- The primary difference between PL and FOL lies in the **ontological commitment** made by each language—that is, what it assumes about the nature of reality. (**Ontological Commitment:** What exists in the world — TRUTH)
- For example, PL assumes that there are facts that either hold or do not hold in the world. Each fact can be in one of two states—true or false—and each model assigns true or false to each proposition symbol.
- FOL, that the world consists of objects with certain relations among them that do or do not hold. The formal models are correspondingly more complicated than those for propositional logic.
- The Secondary difference, a logic can also be characterized by its **epistemological commitments**—the possible states of knowledge that it allows with respect to each fact.(**Epistemological Commitment:** What an agent believes about facts — BELIEF)

Language	Ontological Commitment (What exists in the world)	Epistemological Commitment (What an agent believes about facts)
Propositional logic	facts	true/false/unknown
First-order logic	facts, objects, relations	true/false/unknown
Temporal logic	facts, objects, relations, times	true/false/unknown
Probability theory	facts	degree of belief $\in [0, 1]$
Fuzzy logic	facts with degree of truth $\in [0, 1]$	known interval value

Figure :Formal languages and their ontological and epistemological commitments.

Syntax of first-order logic:

$$
\begin{array}{rcl}
Sentence & \rightarrow & AtomicSentence \mid ComplexSentence \\
AtomicSentence & \rightarrow & Predicate \mid Predicate(Term, \ldots) \mid Term = Term \\
ComplexSentence & \rightarrow & (\, Sentence \,) \\
 & \mid & \neg\, Sentence \\
 & \mid & Sentence \wedge Sentence \\
 & \mid & Sentence \vee Sentence \\
 & \mid & Sentence \;\Rightarrow\; Sentence \\
 & \mid & Sentence \;\Leftrightarrow\; Sentence \\
 & \mid & Quantifier\ Variable, \ldots\ Sentence \\
 & & \\
Term & \rightarrow & Function(Term, \ldots) \\
 & \mid & Constant \\
 & \mid & Variable \\
 & & \\
Quantifier & \rightarrow & \forall \mid \exists \\
Constant & \rightarrow & A \mid X_1 \mid John \mid \cdots \\
Variable & \rightarrow & a \mid x \mid s \mid \cdots \\
Predicate & \rightarrow & True \mid False \mid After \mid Loves \mid Raining \mid \cdots \\
Function & \rightarrow & Mother \mid LeftLeg \mid \cdots \\
\text{OPERATOR PRECEDENCE} & : & \neg, =, \wedge, \vee, \Rightarrow, \Leftrightarrow
\end{array}
$$

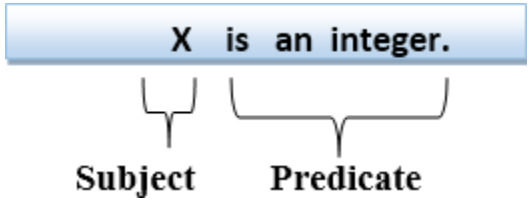
- A sentence in FOL is either an **atomic** sentence or **complex** sentence.

Atomic sentence

- These sentences are formed from a predicate symbol followed by a parenthesis with a sequence of terms.
- Predicate (term1, term2, ..., term n).
- Example: Ravi and Ajay are brothers: => Brothers(Ravi, Ajay), Chinky is a cat: => cat (Chinky).

Complex sentences

- Complex sentences are made by combining atomic sentences using connectives.
- **Consider the statement: "x is an integer.",** it consists of two parts, the first part x is the subject of the statement and second part "is an integer," is known as a predicate.



Representing knowledge in first-order logic

- The objects from the real world are represented by constant symbols (a,b,c,...). For instance, the symbol “Tom” may represent a certain individual called Tom.
- Properties of objects may be represented by predicates applied to those objects (P(a), ...): e.g "male(Tom)" represents that Tom is a male.
- Relationships between objects are represented by predicates with more arguments: "father(Tom, Bob)" represents the fact that Tom is the father of Bob.
- The value of a predicate is one of the boolean constants T (i.e. true) or F (i.e. false)."father(Tom, Bob) = T" means that the sentence "Tom is the father of Bob" is true. "father(Tom, Bob) = F" means that the sentence "Tom is the father of Bob" is false.
- Besides constants, the arguments of the predicates may be functions (f,g,...) or variables (x,y,...).

- Function symbols denote mappings from elements of a domain (or tuples of elements of domains) to elements of a domain. For instance, weight is a function that maps objects to their weight: weight (Tom) = 150.Therefore the predicate greater-than (weight (Bob), 100) means that the weight of Bob is greater than 100. The arguments of a function may themselves be functions.
- Variable symbols represent potentially any element of a domain and allow the formulation of general statements about the elements of the domain.
- The quantifier ∀ and ∃ are used to build new formulas from old ones.

" ∃x P(x)" expresses that there is at least one element of the domain that makes P(x) true.

" ∃ x mother(x, Bob)” means that there is x such that x is mother of Bob or, otherwise stated, Bob has a mother.

" ∀x P(x)" expresses that for all elements of the domain P(x) is true.

Quantifiers:

- FOPL also provides variable quantifiers that allow the expression of properties for entire collections of objects instead of enumerating objects by name.
- Two types of quantifiers:
- Universal quantifier (∀): means —for all.
- Existential quantification (∃): means —there exists.

Universal quantification: (for all, everyone, everything)

- **Syntax:** ∀ <*Variables*> <*sentence*>
- If x is a variable, then ∀x is read as: For all x, For each x, For every x.
- **Example:** Everyone at CAB is smart:

" ∀x At(x, CAB) → Smart(x) (we can also read this as —if X is at CAB, then X is smart)

All man drink coffee:

$\forall x$ man(x) $\rightarrow$ drink (x, coffee)

(we can also read this as -There are all x where x is a man who drink coffee)

Existential quantification: (for some, at least one)

- **Syntax:** $\exists$ <*variables*> <*sentence*>

Example :

- "Everyone at CAB is smart"

$\exists x$ At(x, CAB) $\wedge$ Smart(x)

- If x is a variable, then existential quantifier will be $\exists x$ or $\exists(x)$. And it will be read as: There exists a 'x.', For some 'x.', For at least one 'x.'

- "Some boys are intelligent"

$\exists x$: boys(x) $\wedge$ intelligent(x) -(also read as- There are some x where x is a boy who is intelligent)

Points to remember

- The main connective for universal quantifier $\forall$ is implication $\rightarrow$.
- The main connective for existential quantifier $\exists$ is and $\wedge$.

Properties of Quantifiers

- In universal quantifier, $\forall x \forall y$ is similar to $\forall y \forall x$.
- In Existential quantifier, $\exists x \exists y$ is similar to $\exists y \exists x$.
- $\exists x \forall y$ is not similar to $\forall y \exists x$.

For examples:

- "Brothers are siblings"

$\forall x\ \forall y\ Brother(x,y) \Rightarrow Sibling(x,y)$

- "There is someone who is loved by everyone,"

$\exists y\ \forall x\ Loves(x,y)$.

- “Every man respects his parent”

$\forall x$ man(x) $\rightarrow$ respects (x, parent)

- “Some boys play cricket”

$\exists x$ boys(x) $\rightarrow$ play(x, cricket)

- “Not all students like both Mathematics and Science”

$\neg\forall$ (x) [student(x) $\rightarrow$ like(x, Mathematics) $\wedge$ like(x, Science)]

- “Only one student failed in Mathematics”

$\exists$(x) [student(x) $\rightarrow$ failed (x, Mathematics) $\wedge\forall$ (y) [$\neg$(x==y) $\wedge$ student(y) $\rightarrow$ $\neg$failed (x, Mathematics)]

Translating English to FOL

- Every gardener likes the sun.

  (∀x) gardener(x) => likes(x,Sun)

- Not Every gardener likes the sun.

  ~((∀x) gardener(x) => likes(x,Sun))

- You can fool some of the people all of the time.

  (∃x)(∀t) person(x) ^ time(t) => can-be-fooled(x,t)

- You can fool all of the people some of the time.

  (∀x)(∃t) person(x) ^ time(t) => can-be-fooled(x,t)

  (the time people are fooled may be different)

- You can fool all of the people at some time.

  (∃t)(∀x) person(x) ^ time(t) => can-be-fooled(x,t)

  (all people are fooled at the same time)

- You can not fool all of the people all of the time.

  ~((∀x)(∀t) person(x) ^ time(t) => can-be-fooled(x,t))

- All purple mushrooms are poisonous.

  (∀x) (mushroom(x) ^ purple(x)) => poisonous(x)

- No purple mushroom is poisonous.

  ~(∃x) purple(x) ^ mushroom(x) ^ poisonous(x)

  (∀x) (mushroom(x) ^ purple(x)) => ~poisonous(x)

- There are exactly two purple mushrooms.

  (∃x)(Ey) mushroom(x) ^ purple(x) ^ mushroom(y) ^ purple(y) ^ ~(x=y) ^

  (∀z) (mushroom(z) ^ purple(z)) => ((x=z) v (y=z))

- Clinton is not tall.

  ~tall(Clinton)

- X is above Y if X is directly on top of Y or there is a pile of one or more other objects directly on top of one another starting with X and ending with Y.

  (∀x)(∀y) above(x,y) <=> (on(x,y) v (∃z) (on(x,z) ^ above(z,y)))

Inference with FOPL: By converting into PL (existential and universal instantiation)

- As propositional logic we also have inference rules in first-order logic, so following are some basic inference rules in FOL:
  - Universal Generalization
  - Universal Instantiation
  - Existential Instantiation
  - Existential introduction

Universal Instantiation:

- Universal instantiation is also called as universal elimination or UI is a valid inference rule. It can be applied multiple times to add new sentences.
- The new KB is logically equivalent to the previous KB.
- As per UI, **we can infer any sentence obtained by substituting a ground term for the variable**.
- The UI rule state that we can infer any sentence P(c) by substituting a ground term c (a constant within domain x) from **∀x P(x) for any object in the universe of discourse**.

It can be represented as: $\frac{\forall x\, P(x)}{P(c)}$

Examples:

IF "Every person like ice-cream"=> ∀x P(x) so we can infer that

"John likes ice-cream" => P(c)

Let's take a famous example,

"All kings who are greedy are Evil."

So let our knowledge base contains this detail as in the form of FOL.

$\forall x$ king(x) $\wedge$ greedy (x) $\rightarrow$ Evil (x),

So from this information, we can infer any of the following statements using Universal Instantiation:

King(John) $\wedge$ Greedy (John) $\rightarrow$ Evil (John),

King(Richard) $\wedge$ Greedy (Richard) $\rightarrow$ Evil (Richard),

King(Father(John)) $\wedge$ Greedy (Father(John)) $\rightarrow$ Evil (Father(John)),

Existential Instantiation:

- Existential instantiation is also called as Existential Elimination, which is a valid inference rule in first-order logic.
- It can be applied only once to replace the existential sentence.
- The new KB is not logically equivalent to old KB, but it will be satisfiable if old KB was satisfiable.
- This rule states that one can infer P(c) from the formula given in the form of ∃x P(x) for a new constant symbol c.
- The restriction with this rule is that c used in the rule must be a new term for which P(c) is true.
- It can be represented as: $\frac{\exists x\, P(x)}{P(c)}$

Example:

From the given sentence: **∃x Crown(x) ∧ OnHead(x, John),**

So we can infer: **Crown(K) ∧ OnHead(K, John),** as long as K does not appear in the knowledge base.

- The above used K is a constant symbol, which is called **Skolem constant**.
- The Existential instantiation is a special case of **Skolemization process**.

Generalized Modus Ponens Rule:

- For the inference process in FOL, we have a single inference rule which is called Generalized Modus Ponens. It is **lifted version of Modus ponens**.

[Generalized Modus Ponens = lifted Modus Ponens

Lifted- transformed from:

Propositional Logic →First-order Logic]

- Generalized Modus Ponens can be summarized as, " P implies Q and P is asserted to be true, therefore Q must be True.“
- According to Modus Ponens, for atomic sentences **pi, pi', q**. Where there is a substitution θ such that SUBST **(θ, pi',) = SUBST(θ, pi)**, it can be represented as:

$$\frac{p1', p2', \ldots, pn', (p1 \wedge p2 \wedge \ldots \wedge pn \Rightarrow q)}{SUBST(\theta, q)}$$

(Note: Backwards chaining, forwards chaining, and resolution algorithms also have lifted forms you will see later)

Example:

We will use this rule for Kings are evil, so we will find some x such that x is king, and x is greedy so we can infer that x is evil.

Here let say,

p1' is king(John) p1 is king(x)

p2' is Greedy(y) p2 is Greedy(x)

θ is {x/John, y/John} q is evil(x)

SUBST(θ,q) is Evil(John).

Unification:

- Lifted inference rules require finding substitutions that make different logical expressions look identical. This process is called unification and is a key component of all first-order inference algorithms.
- The UNIFY algorithm takes two sentences and returns a unifier for them (a substitution) if one exists:

UNIFY(p, q)=θ where SUBST(θ, p)=SUBST(θ, q) .

- Suppose we have a query AskVars(Knows(John, x)): whom does John know? Answers to this query can be found by finding all sentences in the knowledge base that unify with Knows(John, x). Here are the results of unification with four different sentences that might be in the knowledge base:

UNIFY(Knows(John, x), Knows(John, Jane)) = {x/Jane}

UNIFY(Knows(John, x), Knows(y, Bill)) = {x/Bill, y/John}

UNIFY(Knows(John, x), Knows(y, Mother(y))) = {y/John, x/Mother(John)}

UNIFY(Knows(John, x), Knows(x, Elizabeth)) = failure .

Following are some basic conditions for unification:

- Predicate symbol must be same, atoms or expression with different predicate symbol can never be unified.
- Number of Arguments in both expressions must be identical.
- Unification will fail if there are two similar variables present in the same expression.

- Three major families of first-order inference algorithms:
  - **Forward chaining**
  - **Backward chaining**
  - **Resolution**

Resolution

- It was invented by a Mathematician John Alan Robinson in the year 1965.
- Resolution is used, if there are various statements are given, and we need to prove a conclusion of those statements. Unification is a key concept in proofs by resolutions.
- Resolution is a single inference rule which can efficiently operate on the **conjunctive normal form or clausal form**.
- **Clause**: Disjunction of literals (an atomic sentence) is called a **clause**. It is also known as a unit clause.
- **Conjunctive Normal Form**: A sentence represented as a conjunction of clauses is said to be **conjunctive normal form** or **CNF**.

The resolution inference rule:

- The resolution rule for first-order logic is simply a lifted version of the propositional rule. Resolution can resolve two clauses if they contain complementary literals, which are assumed to be standardized apart so that they share no variables.

$$\frac{l_1 \vee \ldots\ldots \vee l_k, \qquad m_1 \vee \ldots\ldots \vee m_n}{\mathrm{SUBST}(\theta, l_1 \vee \ldots\ldots \vee l_{i-1} \vee l_{i+1} \vee \ldots \vee l_k \vee m_1 \vee \ldots\ldots \vee m_{j-1} \vee m_{j+1} \vee \ldots \vee m_n)}$$

Where l_i and m_j are complementary literals.

This rule is also called the **binary resolution rule** because it only resolves exactly two literals.

Example:

We can resolve two clauses which are given below:

[Animal (g(x) V Loves (f(x), x)] and [¬ Loves(a, b) V ¬ Kills(a, b)]

Where two complimentary literals are: Loves (f(x), x) and ¬ Loves (a, b)

Steps for Resolution

1. **Conversion of facts into first-order logic**
2. **Convert FOL statements into CNF** (Eliminate all implication (→) and rewrite, Move negation (¬)inwards and rewrite, Rename variables or standardize variables, Eliminate existential instantiation quantifier by elimination, Drop Universal quantifiers, Distribute conjunction ∧ over disjunction ¬)
3. **Negate the statement which needs to prove (proof by contradiction)**
4. **Draw resolution graph (unification)**

Proof By Resolution In FOL Examples

Resolution in FOL: Example

Example:

a. Rimi is Hungry
b. If Rimi is hungry she barks
c. If Rimi is barking then Raja is angry

Prove by resolution that:

Raja is angry

Step-1: Conversion of Facts into FOL

In the first step we will convert all the given statements into its first orde logic.

a. Hungry(Rimi)
b. hungry(Rimi) ->barks(Rimi)
c. barks(Rimi) ->angry(Raja)

Step-2: Conversion of FOL into CNF

In First order logic resolution, it is required to convert the FOL into CNF as CNF form makes easier for resolution proofs.

a. Hungry(Rimi)
b. ~hungry(Rimi) V barks(Rimi)
c. ~barks(Rimi) V angry(Raja)

Step-3: Negate the statement to be proved

- In this statement, we will apply negation to the conclusion statements, which will be written as

~Angry(Raja

Step-4: Draw Resolution graph:

- Now in this step, we will solve the problem by resolution tree using substitution. For the above problem, it will be given as follows:

- Hence the negation of the conclusion has been proved as a complete contradiction with the given set of statements.

a. Hungry(Rimi)
b. ~hungry(Rimi) V barks(Rimi)
c. ~barks(Rimi) V angry(Raja)

~Angry(Raja

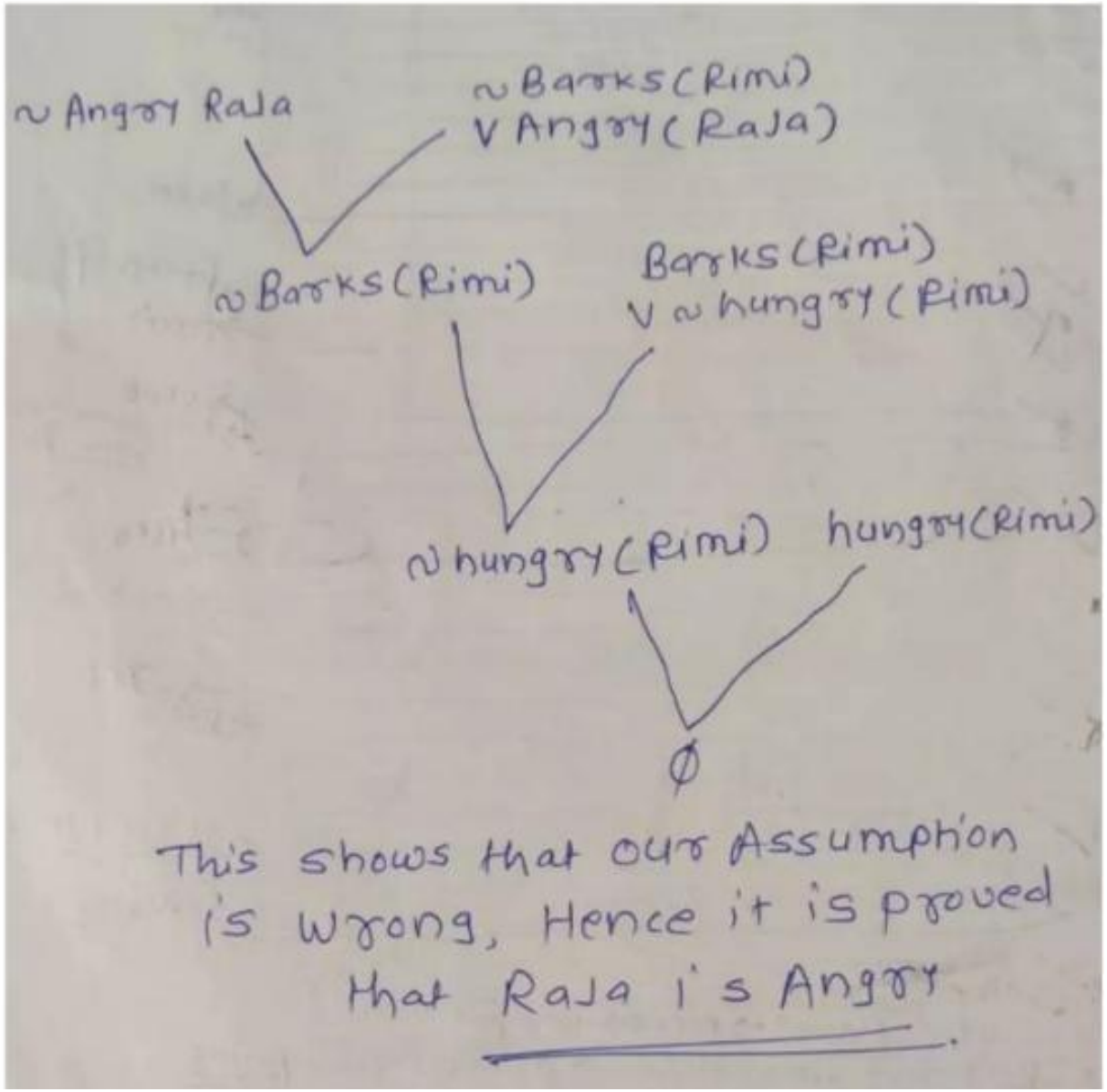
Step-4: Draw Resolution graph:

a. **Hungry(Rimi)**

b. **~hungry(Rimi) V barks(Rimi)**

c. **~barks(Rimi) V angry(Raja)**

~Angry(Raja



Proof By Resolution In FOL Examples

Resolution in FOL: Example

Example:

a. All people who are graduating are happy
b. All happy people smile
c. Someone is graduating

Prove by resolution that: Someone is smiling

Step-1: Conversion of Facts into FOL

In the first step we will convert all the given statements into its first order logic. ∀x∃x

a. ∀x :graduating(x)->happy(x)
b. ∀x: happy(x) ->smile(x)
c. ∃x :graduating(x)

∃x:smiling(x)

Step-2: Conversion of FOL into CNF

In First order logic resolution, it is required to convert the FOL into CNF as CNF form makes easier for resolution proofs.

a. ~graduating(x) V happy(x)
b. ~happy(x1) V smile(x1)
c. graduating(x2)

smiling(x3)

Step-3: Negate the statement to be proved

- In this statement, we will apply negation to the conclusion statements, which will be written as

~smiling(x3)

Step-4: Draw Resolution graph:

- Now in this step, we will solve the problem by resolution tree using substitution. For the above problem, it will be given as follows:

- Hence the negation of the conclusion has been proved as a complete contradiction with the given set of statements.

a. ~graduating(x) V happy(x)
b. ~happy(x1) V smile(x1)
c. graduating(x2)

~smiling(x3)

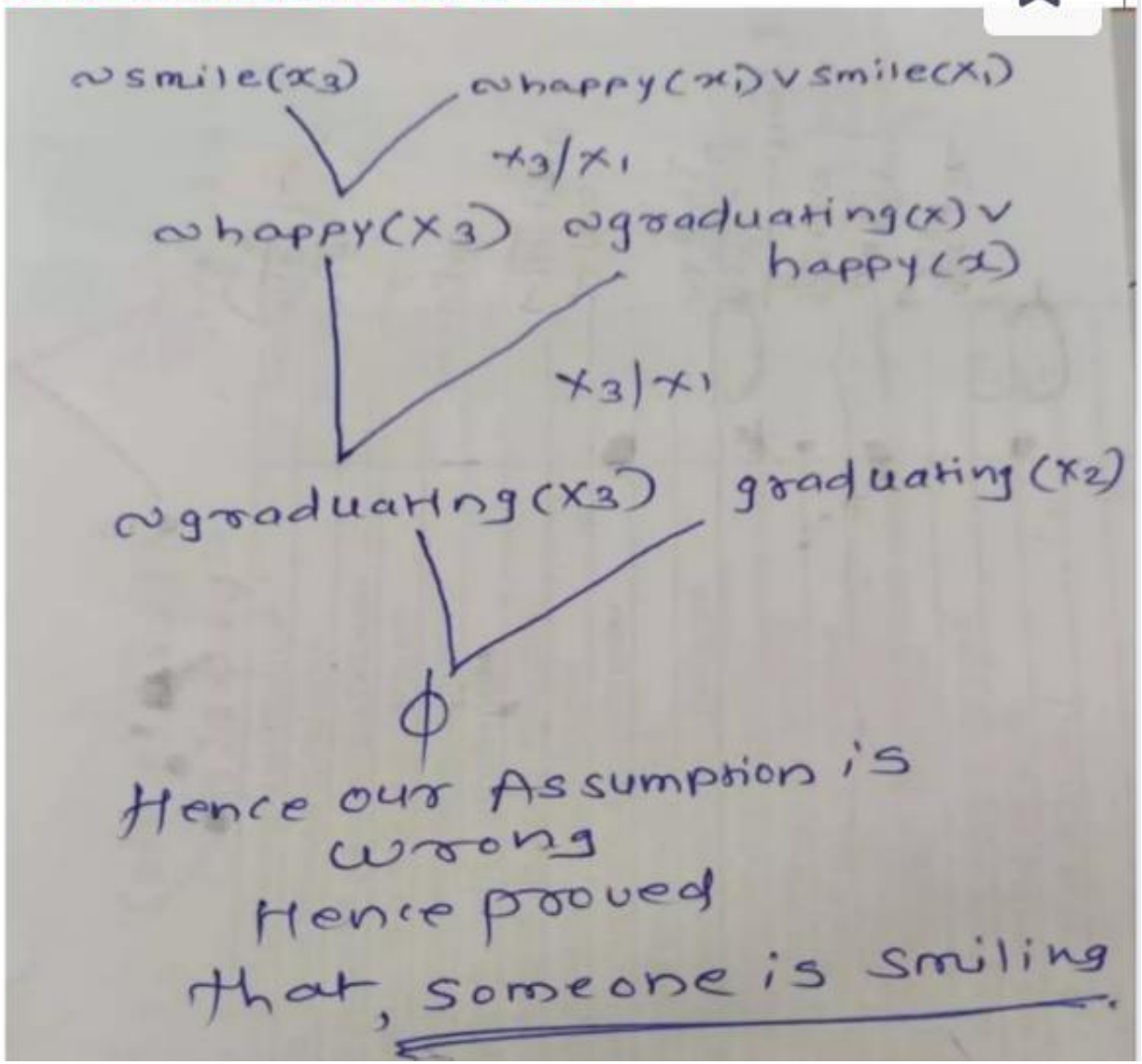
Step-4: Draw Resolution graph:

a. **~graduating(x) V happy(x)**

b. **~happy(x1) V smile(x1)**

c. **graduating(x2)**

~smiling(x3)



Example:

- John likes all kind of food.
- Apple and vegetable are food
- Anything anyone eats and not killed is food.
- Anil eats peanuts and still alive
- Harry eats everything that Anil eats.

Prove by resolution that: John likes peanuts.

Step-1: Conversion of Facts into FOL

a. $\forall x$: food(x) → likes(John, x)

b. food(Apple) ∧ food(vegetables)

c. $\forall x\ \forall y$: eats(x, y) ∧ ¬ killed(x) → food(y)

d. eats (Anil, Peanuts) ∧ alive(Anil).

e. $\forall x$: eats(Anil, x) → eats(Harry, x)

f. $\forall x$: ¬ killed(x) → alive(x) } **added predicates.**

g. $\forall x$: alive(x) → ¬ killed(x)

h. likes(John, Peanuts)

Step-2: Conversion of FOL into CNF

- **Eliminate all implication (→) and rewrite**
  - ∀x ¬ food(x) V likes(John, x)
  - food(Apple) Λ food(vegetables)
  - ∀x ∀y ¬ [eats(x, y) Λ ¬ killed(x)] V food(y)
  - eats (Anil, Peanuts) Λ alive(Anil)
  - ∀x ¬ eats(Anil, x) V eats(Harry, x)
  - ∀x¬ [¬ killed(x)] V alive(x)
  - ∀x ¬ alive(x) V ¬ killed(x)
  - likes(John, Peanuts).

- **Move negation (¬)inwards and rewrite**
  - ∀x ¬ food(x) V likes(John, x)
  - food(Apple) Λ food(vegetables)
  - ∀x ∀y ¬ eats(x, y) V killed(x) V food(y)
  - eats (Anil, Peanuts) Λ alive(Anil)
  - ∀x ¬ eats(Anil, x) V eats(Harry, x)
  - ∀x ¬killed(x)] V alive(x)
  - ∀x ¬ alive(x) V ¬ killed(x)
  - likes(John, Peanuts).

- **Rename variables or standardize variables**
  - ∀x ¬ food(x) V likes(John, x)
  - food(Apple) Λ food(vegetables)
  - ∀y ∀z ¬ eats(y, z) V killed(y) V food(z)
  - eats (Anil, Peanuts) Λ alive(Anil)
  - ∀x ¬ eats(Anil, x) V eats(Harry, x)
  - ∀x ¬killed(x)] V alive(x)
  - ∀x ¬ alive(x) V ¬ killed(x)
  - likes(John, Peanuts).

- **Eliminate existential instantiation quantifier by elimination**
- We will eliminate existential quantifiers in this step, which is referred to as **Skolemization**. However, because there is no existential quantifier in this example problem, all of the assertions in this phase will be the same

- **Drop Universal quantifiers.** We'll remove all universal quantifiers ∃ in this phase because none of the statements are implicitly quantified, therefore we don't need them.

  - ¬ food(x) V likes(John, x)
  - food(Apple)
  - food(vegetables)
  - ¬ eats(y, z) V killed(y) V food(z)
  - eats (Anil, Peanuts)
  - alive(Anil)
  - ¬ eats(Anil, w) V eats(Harry, w)
  - killed(g) V alive(g)
  - ¬ alive(k) V ¬ killed(k)
  - likes(John, Peanuts).

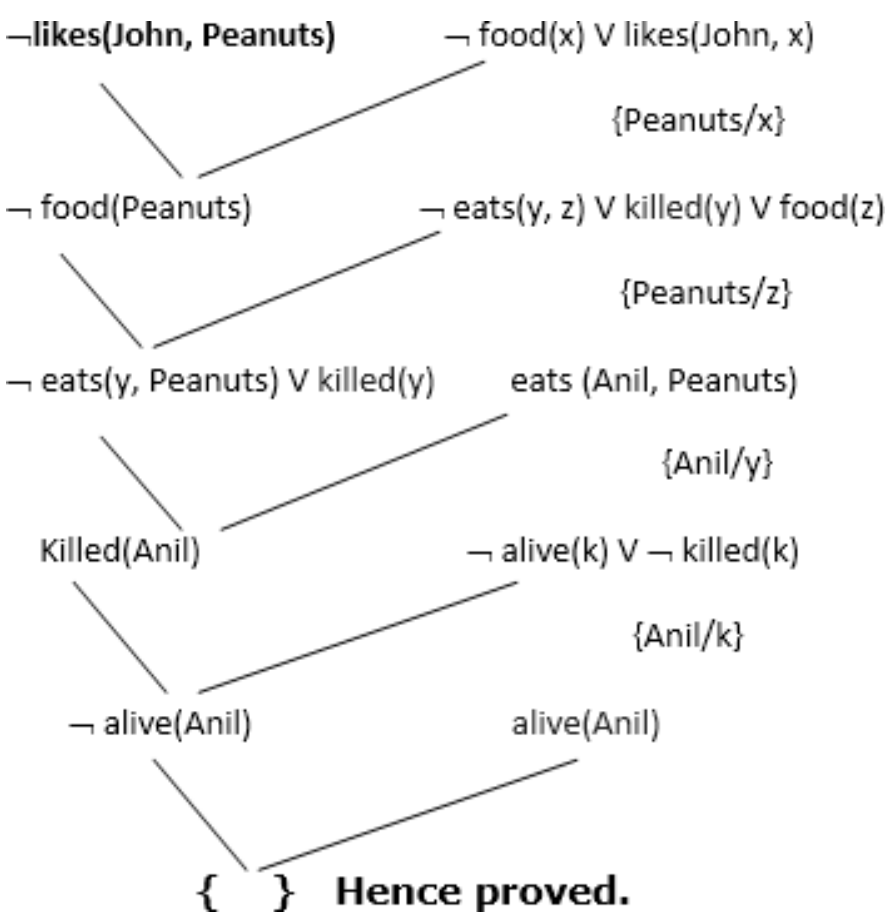
Note: Statements "food(Apple) Λ food(vegetables)" and "eats (Anil, Peanuts) Λ alive(Anil)" can be written in two separate statements.

- **Distribute conjunction ∧ over disjunction ¬.**
- This step will not make any change in this problem.

Step-3: Negate the statement to be proved

In this statement, we will apply negation to the conclusion statements, which will be written as ¬likes(John, Peanuts)

Step-4: Draw Resolution graph:



Hence the negation of the conclusion has been proved as a complete contradiction with the given set of statements.

Handling Uncertain Knowledge

- Till now, we have learned knowledge representation using first-order logic and propositional logic with certainty, which means we were sure about the predicates. With this knowledge representation, we might write A→B, which means if A is true then B is true, but consider a situation where we are not sure about whether A is true or not then we cannot express this statement, this situation is called **uncertainty.**
- FOL can fail for essentially three reasons:
  - **Laziness:** it is not feasible to assemble a set of rules that is sufficiently exhaustive. If we could, it would not be feasible to apply them.
  - **Theoretical ignorance:** insufficient knowledge exists to allow us to write the rules.
  - **Practical ignorance:** even if the rules have been obtained there may be insufficient information to apply them.

- So to represent uncertain knowledge, where we are not sure about the predicates, we need **uncertain reasoning or probabilistic reasoning.**

Causes of uncertainty:

Following are some leading causes of uncertainty to occur in the real world.

- Information occurred from unreliable sources.
- Experimental Errors
- Equipment fault
- Temperature variation
- Climate change.

Probabilistic reasoning:

- Probabilistic reasoning is a way of knowledge representation where we apply the concept of probability to indicate the uncertainty in knowledge.

- In probabilistic reasoning, we combine probability theory with logic to handle the uncertainty. We use probability in probabilistic reasoning because it provides a way to handle the uncertainty that is the result of someone's laziness and ignorance.
- In the real world, there are lots of scenarios,
  - "It will rain today,"
  - "behavior of someone for some situations,"
  - "A match between two teams or two players."
- In probabilistic reasoning, there are two ways to solve problems with uncertain knowledge:
  - Bayes' rule
  - Bayesian Statistics.

Probability:

- Probability can be defined as a chance that an uncertain event will occur. It is the numerical measure of the likelihood that an event will occur. The value of probability always remains between 0 and 1 that represent ideal uncertainties.

1. $0 \leq P(A) \leq 1$, where P(A) is the probability of an event A.
2. P(A) = 0, indicates total uncertainty in an event A.
3. P(A) =1, indicates total certainty in an event A.

- We can find the probability of an uncertain event by using the below formula.

$$\textbf{Probability of occurrence} = \frac{\text{Number of desired outcomes}}{\text{Total number of outcomes}}$$

- P(¬A) = probability of a not happening event.
- P(¬A) + P(A) = 1.

Random Variables:

- In probability theory and statistics, a **random variable** (or **stochastic variable**) is a way of assigning a value (often a real number) to each possible outcome of a random event.
- These values might represent the possible outcomes of an experiment, or the potential values of a quantity whose value is uncertain (e.g., as a result of incomplete information or imprecise measurements.)
- Random variables are usually real valued, but one can consider arbitrary types such as boolean values, complex numbers, vectors, matrices, sequences, trees, sets, shapes, manifolds and functions. The term random element is used to encompass all such related concepts.
- For example: There are two possible outcomes for a coin toss: heads, or tails. The possible outcomes for one fair coin toss can be described using the following random variable:

$$X = \begin{cases} \text{head}, \\ \text{tail}. \end{cases}$$

and if the coin is equally likely to land on either side then it has a probability mass function given by:

$$\rho_X(x) = \begin{cases} \frac{1}{2}, & \text{if } x = \text{head}, \\ \frac{1}{2}, & \text{if } x = \text{tail}. \end{cases}$$

Example:

- Cavity (do I have a cavity ?) is one of <cavity, not cavity>
- Weather is one of <sunny, rainy, cloudy, snow>

Both Cavity and Weather are discrete random variables

Prior (Unconditional) probability:

- A prior probability denotes the probability (degree of belief) assigned to a proposition in the absence of any other evidence.

Example 1:

- Pr(Cavity = true) = 0.05, denotes the degree of belief that a random person has a cavity before we make any actual observation of that person.
- To keep things compact, we will use Pr(Cavity)
- to denote the entire probability distribution of the random variable Cavity.

Instead of

Pr(Cavity = true) = 0.05,

Pr(Cavity = false) = 0.95

write

Pr(Cavity) = (0.05, 0.95)

Example :

P(Weather= sunny) = 0.72, P(Weather= rain) = 0.1, P(Weather= cloudy) = 0.08, P(Weather= snow) = 0.1 {sum is 1}

Probability distribution gives values for all possible assignments: P(Weather) = (0.72, 0.1, 0.08, 0.1)

Joint probability distribution for a set of r.v.s gives the probability of every atomic event on those r.v.s (i.e., every sample point).

P(Weather, Cavity) = a 4 ×2 matrix of values.

Weather =	Sunny	Rain	Cloudy	Snow
Cavity=true	0.144	0.02	0.016	0.02
Cavity=false	0.576	0.08	0.064	0.08

Posterior (Conditional) probability:

- Denotes a degree of belief after evidence is perceived.

or

- Conditional probability is a probability of occurring an event when another event has already happened.
- Let's suppose, we want to calculate the event A when event B has already occurred, "the probability of A under the conditions of B", it can be written as:

$$P(A|B) = \frac{P(A \wedge B)}{P(B)}$$

Where, $P(A \wedge B)$= Joint probability of A and B

P(B)= Marginal probability of B.

- If the probability of A is given and we need to find the probability of B, then it will be given as:

$$P(B|A) = \frac{P(A \wedge B)}{P(A)}$$

Example:

In a class, there are 70% of the students who like English and 40% of the students who likes English and mathematics, and then what is the percent of students those who like English also like mathematics?

Solution:

Let, A is an event that a student likes Mathematics

B is an event that a student likes English.

$$P(A|B) = \frac{P(A \wedge B)}{P(B)} = \frac{0.4}{0.7} = 57\%$$

Hence, 57% are the students who like English also like Mathematics.

Inference using Full Joint Distribution:

- We use the full joint distribution as the knowledge base from which answers to all questions may be derived. The probability of a proposition is equal to the sum of the probabilities of the atomic events in which it holds.

$P(a) = \Sigma P(e_i)$

Therefore, given a full joint distribution that specifies the probabilities of all the atomic events, one can compute the probability of any proposition.

Full Joint probability distribution : an example

- We consider the following domain consisting of three Boolean variables: Toothache, Cavity, and Catch (the dentist‘s nasty steel probe catches in my tooth).
- The full joint distribution is the following 2x2x2 table:

	toothache		¬toothache	
	catch	¬catch	catch	¬catch
cavity	0.108	0.012	0.072	0.008
¬cavity	0.016	0.064	0.144	0.576

- The probability of any proposition can be computed from the probabilities in the table. The probabilities in the joint distribution must sum to 1.
- Each cell represents an atomic event and these are all the possible atomic events.

P(toothache) = 0.108 + 0.012 + 0.016 + 0.064 = 0.2

P(cavity ∨ toothache)=P(cavity, toothache, catch) + P(cavity, toothache, ¬catch) +P(cavity, ¬toothache, catch) + P(cavity, ¬toothache, ¬catch) + P(¬cavity, toothache, catch) + P(¬cavity, toothache, ¬catch)

= 0.108+0.012+0.072+0.008+0.016+0.064=0.28

We simply identify those atomic events in which the proposition is true and add up their probabilities

Bayes' Theorem:

- Bayes' theorem is also known as **Bayes' rule, Bayes' law**, or **Bayesian reasoning**, which determines the probability of an event with uncertain knowledge.
- In probability theory, it relates the conditional probability and marginal probabilities of two random events.
- Bayes' theorem was named after the British mathematician **Thomas Bayes**. The **Bayesian inference** is an application of Bayes' theorem, which is fundamental to Bayesian statistics.
- Bayes' theorem can be derived using product rule and conditional probability of event A with known event B:

As from product rule we can write: $P(A \wedge B) = P(A|B)\, P(B)$ or

Similarly,

the probability of event B with known event A: $P(A \wedge B) = P(B|A)\, P(A)$

Equating right hand side of both the equations, we will get:

$$P(A|B) = \frac{P(B|A)\,P(A)}{P(B)} \quad \text{....(a)}$$

- The above equation (a) is called as **Bayes' rule** or **Bayes' theorem**. This equation is basic of most modern AI systems for **probabilistic inference**.
- It shows the simple relationship between joint and conditional probabilities. Here,
- P(A|B) is known as **posterior**, which we need to calculate, and it will be read as Probability of hypothesis A when we have occurred an evidence B.
- P(B|A) is called the likelihood, in which we consider that hypothesis is true, then we calculate the probability of evidence.
- P(A) is called the **prior probability**, probability of hypothesis before considering the evidence
- P(B) is called **marginal probability**, pure probability of an evidence.
- In the equation (a), in general, we can write P (B) = P(A)*P(B|A_i), hence the Bayes' rule can be written as:

$$P(A_i|B) = \frac{P(A_i)*P(B|A_i)}{\sum_{i=1}^{k} P(A_i)*P(B|A_i)}$$

Where A_1, A_2, A_3,....., A_n is a set of mutually exclusive and exhaustive events.

Why is the Bayes' rule is useful in practice?

- Bayes' rule is useful in practice because there are many cases where we have good probability estimates for three of the four probabilities involved, and therefore can compute the fourth one.
- Suppose we want to perceive the effect of some unknown cause, and want to compute that cause, then the Bayes' rule becomes:

$$\mathbf{P(cause|effect)} = \frac{P(effect|cause)\ P(cause)}{P(effect)}$$

Q. A doctor knows that the disease meningitis causes the patient to have a stiff neck 80% of the time. The doctor also knows that the probability that a patient has meningitis is 1/30,000, and the probability that any patient has a stiff neck is 2%. Find the probability that a patient with a stiff neck has meningitis.

Here, we are given;

p(a|b) = 0.8

p(b) = 1/30000

p(a) = 0.2

Now using Bayes‘ rule;

P(b|a) = P(a|b) P(b)/P(a)

= (0.8*1/30000)/(0.02) = 0.001333333.

Applications of Bayes' theorem:

- It is used to calculate the next step of the robot when the already executed step is given.
- Bayes' theorem is helpful in weather forecasting.
- It can solve the Monty Hall problem.

Bayesian Networks:

- It is also called a **Bayes network, belief network, decision network**, or **Bayesian model**.
- "A Bayesian network is a probabilistic graphical model which represents a set of variables and their conditional dependencies using a directed acyclic graph."

or

- A Bayesian network is a directed graph in which each node is annotated with quantitative probability information. The full specification is as follows:

1. Each node corresponds to a random variable, which may be discrete or continuous.
2. Directed links or arrows connect pairs of nodes. If there is an arrow from node X to node Y, X is said to be a parent of Y. The graph has no directed cycles and hence is a directed acyclic graph, or DAG.
3. Each node X_i has associated probability information $\theta(X_i \mid Parents(X_i))$ that quantifies Parameter the effect of the parents on the node using a finite number of parameters.

- It can also be used in various tasks including prediction, anomaly detection, diagnostics, automated insight, reasoning, time series prediction, and decision making under uncertainty.
- The **intuitive** meaning of an arrow is typically that X has a direct influence on Y.
- The Bayesian Network is made up of two pieces that can be used to create models from data and expert opinions:
  - Directed Acyclic Graph.
  - Table of conditional probabilities.
- Our existing simple world, consisting of the variables Toothache, Cavity, Catch, and Weather. Nodes and Arcs (directed linkages) make form a Bayesian network graph as;

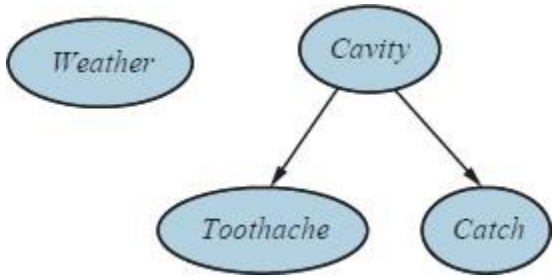


Figure : A simple Bayesian network in which Weather is independent of the other three variables and Toothache and Catch are conditionally independent, given Cavity.

Example:

You have a new burglar alarm installed at home. It is fairly reliable at detecting a burglary, but is occasionally set off by minor earthquakes. You also have two neighbors, John and Mary, who have promised to call you at work when they hear the alarm. John nearly always calls when he hears the alarm, but sometimes confuses the telephone ringing with the alarm and calls then, too. Mary, on the other hand, likes rather loud music and often misses the alarm altogether.

We would like have to estimate the probability of a burglary with given evidence who has or has not call.

Here,

Variables: Burglary, Earthquake, Alarm, JohnCalls, MaryCalls (Burglary and Earthquakes directly affect the probability of the Alarm's going off, but whether John and Mary call depends only on the alarm)

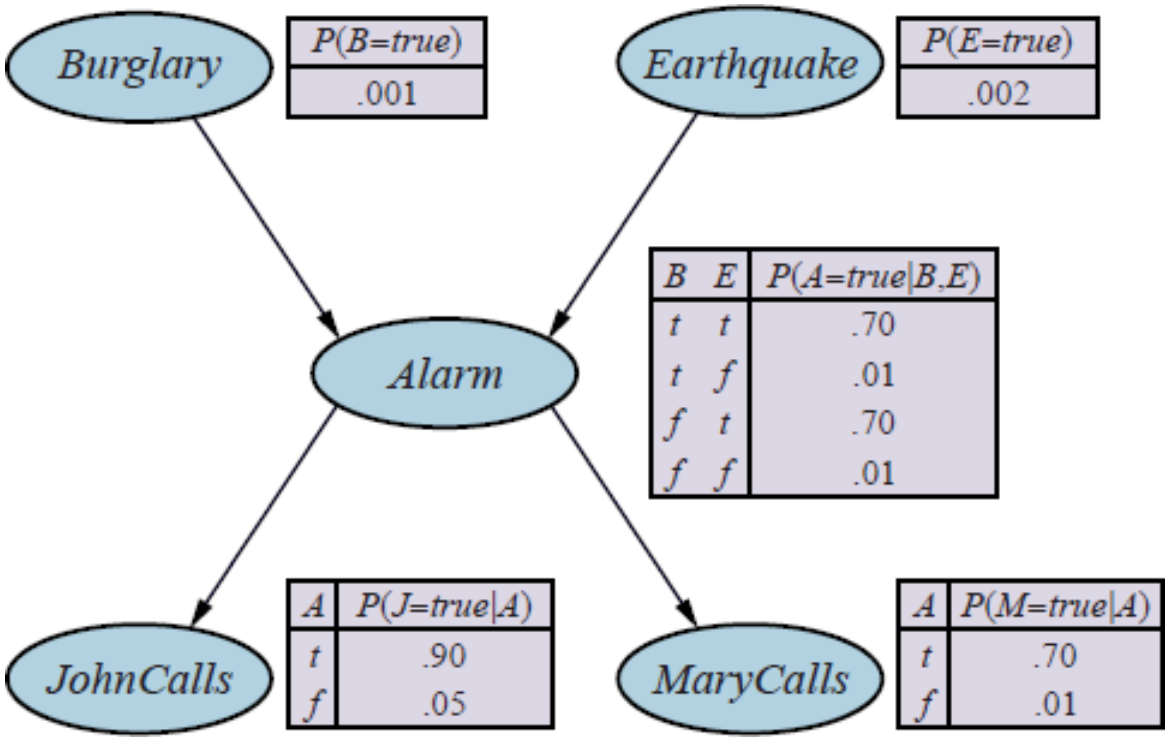


Figure : A typical Bayesian network, showing both the topology and the conditional probability tables (CPTs). In the CPTs, the letters *B*, *E*, *A*, *J*, and *M* stand for *Burglary*, *Earthquake*, *Alarm*, *JohnCalls*, and *MaryCalls*, respectively.

Let's take the observed probability for the Burglary and earthquake component:

P(B= True) = 0.001, which is the probability of burglary.

P(B= False)= 0.999, which is the probability of no burglary.

P(E= True)= 0.002, which is the probability of a minor earthquake

P(E= False)= 0.998, Which is the probability that an earthquake not occurred.

We can provide the conditional probabilities as per the below tables:

Conditional probability table for Alarm A: The Conditional probability of Alarm A depends on Burglar and earthquake

B	E	P(A= True)	P(A= False)
True	True	0.70	0.30
True	False	0.01	0.99
False	True	0.70	0.30
False	False	0.01	0.99

Conditional probability table for JohnCalls: The Conditional probability of JohnCalls that he will call depends on the probability of Alarm.

A	P(J= True)	P(J= False)
True	0.90	0.10
False	0.05	0.95

Conditional probability table for MarryCalls: The Conditional probability of MarryCalls that she calls is depending on its Parent Node "Alarm."

A	P(M= True)	P(M= False)
True	0.70	0.30
False	0.01	0.99

From the formula of joint distribution, we can write the problem statement in the form of probability distribution:

$P(j, m, a, \neg b, \neg e) = P(j \mid a)\, P(m \mid a)\, P(a \mid \neg b \wedge \neg e)\, P(\neg b)\, P(\neg e)$

$= 0.90 \times 0.70 \times 0.01 \times 0.999 \times 0.998 = 0.00628$.

Fuzzy Logic(FL)

- The '**Fuzzy**' word means the things that are not clear or are vague. Sometimes, we cannot decide in real life that the given problem or statement is either true or false.

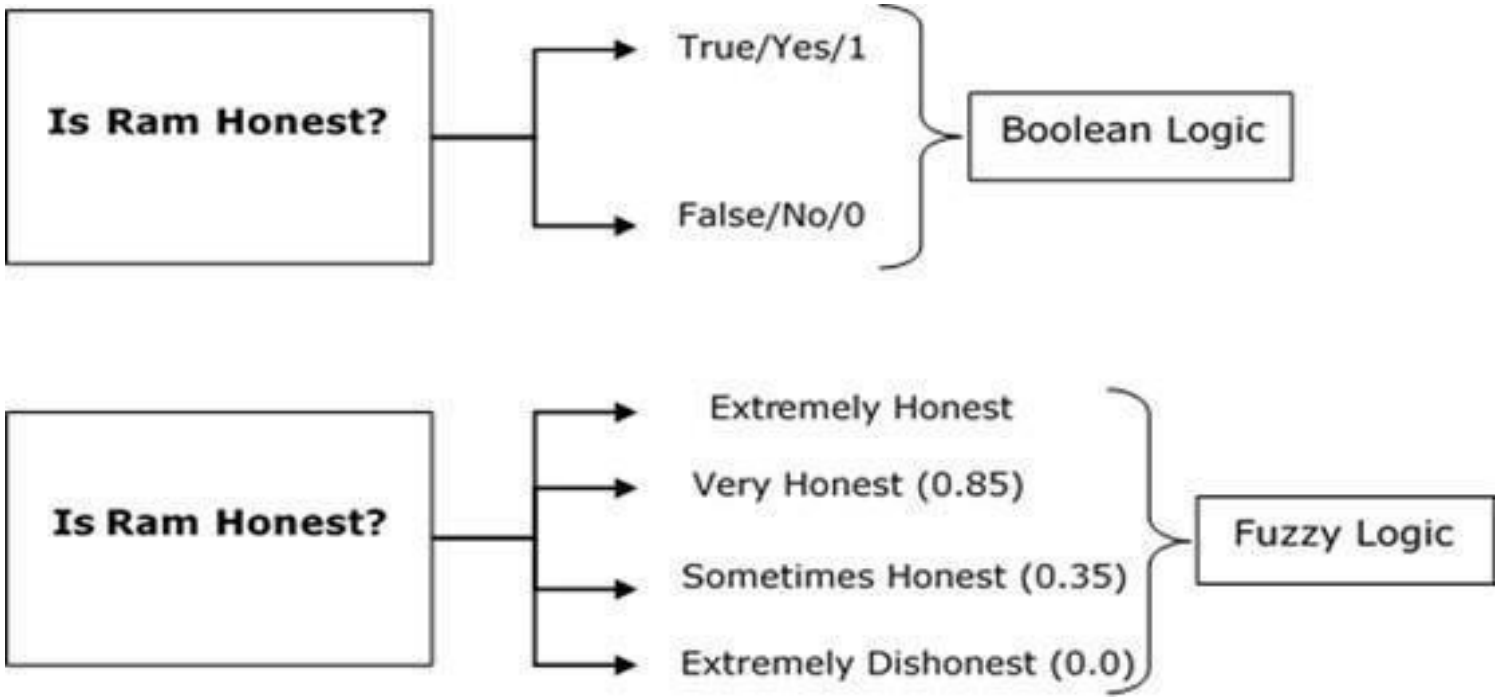
  -E.g., tall, small, good,…. etc.

- Any event, process, or function that is changing continuously cannot always be defined as either true or false.
- such activities can be defined in a Fuzzy manner by using fuzzy logic.

What is Fuzzy Logic?

- fuzzy logic (by Lofti Zadeh in 1965) is not logic that is fuzzy, but logic that is used to describe fuzziness.
- i.e., It deals with vague and imprecise information.
- based on degrees of truth rather than usual true/false or 1/0 like Boolean logic.

Example of Fuzzy Logic as comparing to Boolean Logic



Why do we use Fuzzy Logic?

Generally, we use the fuzzy logic system for both commercial and practical purposes such as:

- It **controls machines** and **consumer products**
- If not accurate reasoning, it at least provides **acceptable reasoning**
- This helps in dealing with the **uncertainty in engineering**

Crisp Set:

- A crisp set is an unordered collection of different elements.
- Represented by listing all the elements comprising it.
- E.g.,

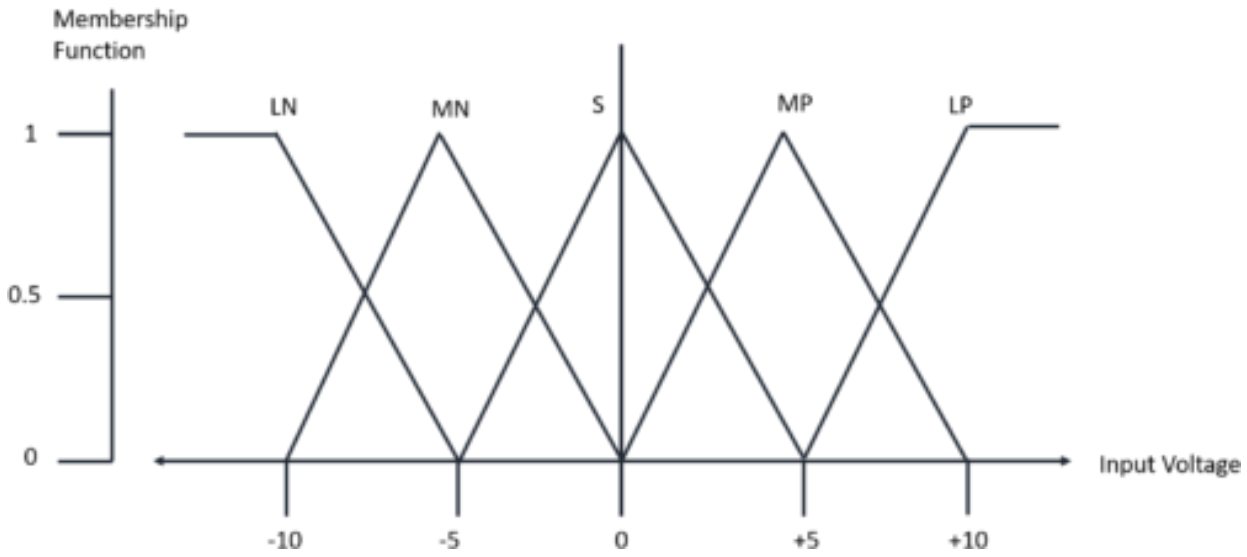
  -Set of vowels in English alphabet, A = {a, e, i, o, u}

  -Set of odd numbers less than 10, B = {1,3,5,7,9}.
- In crisp set element is either the member of a set or not. i.e., membership of elements is either 100%(1) or 0% (0).

Membership Function:

- The membership function is a **graph** that defines how each point in the **input space** is mapped to membership value between 0 and 1.
- It allows you to **quantify linguistic terms** and represent a fuzzy set graphically.
- A membership function for a fuzzy set A on the universe of discourse X is defined as $\boldsymbol{\mu_A: X \rightarrow [0,1]}$
- It quantifies the degree of membership of the element in X to the fuzzy set A.
  - **x-axis** represents the universe of discourse.
  - **y-axis** represents the degrees of membership in the [0, 1] interval.

The membership functions for **LP, MP, S, MN, and LN** are:



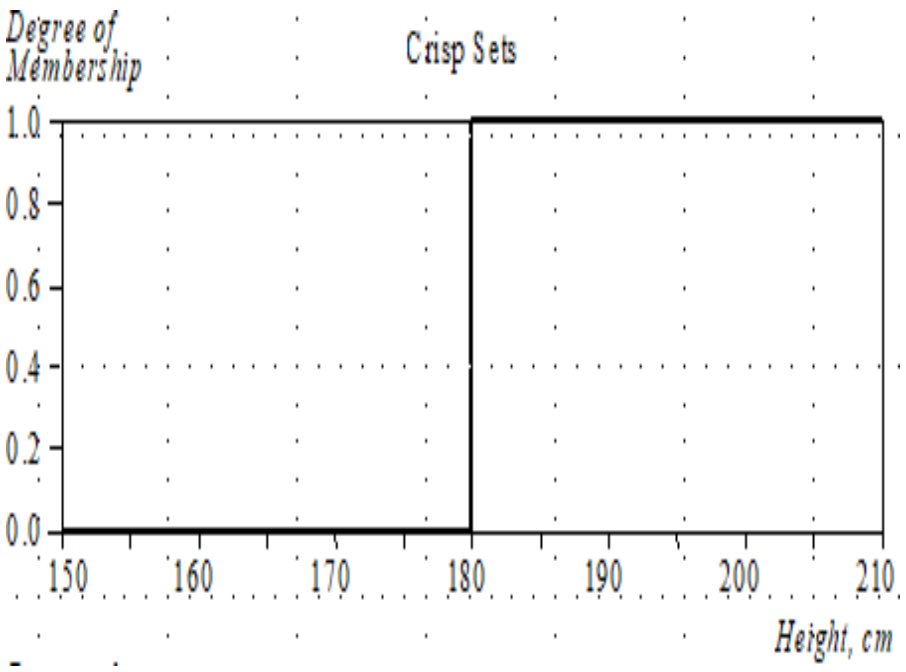
LP	X is Large Positive
MP	X is Medium Positive
S	Small
MN	X is Medium Negative
LN	X is Large Negative

- The triangular membership function shapes are most common among various other membership function shapes. Here, the input to 5-level fuzzifier varies from **-10 volts to +10 volts**. Hence the corresponding output also changes.

- Example of Crisp sets (set of tall men): if Height >180 then men is tall

- Membership on crisp set based on above rule is as shown in table below:

Name	Height, cm	*Crisp*
Chris	208	1
Mark	205	1
John	198	1
Tom	181	1
David	179	0
Mike	172	0
Bob	167	0
Steven	158	0
Bill	155	0
Peter	152	0



Fuzzy Set:

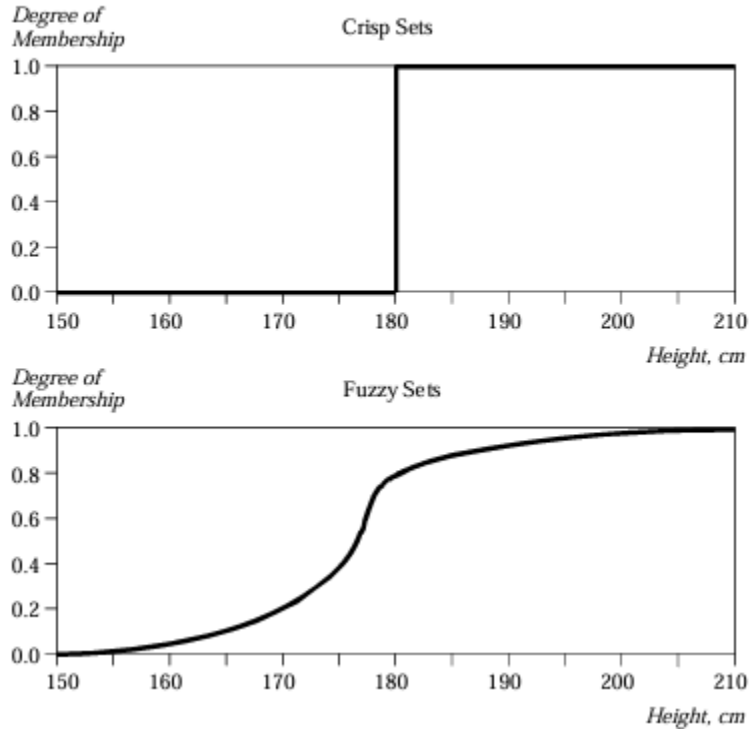
- Elements are allowed to be partially included in the set.
- i.e., A Fuzzy Set has Fuzzy Boundaries
- i.e., membership of elements can be 100% (1) to 0% (0).
- A natural language expresses sets. For example, a *car* indicates a *set of cars*. When we say *a car*, we mean one out of the set of cars.
- Let X be the universe of discourse and its elements be denoted as x. In the classical set theory, crisp set A of X is defined as function $f_A(x)$called the characteristic function of A

$$f_A(x): X \rightarrow \{0, 1\}, \text{ where } \boxed{f_A(x) = \begin{cases} 1, \text{if } x \in A \\ 0, \text{if } x \notin A \end{cases}}$$

- The classical example in fuzzy sets is tall men. The elements of the fuzzy set “tall men” are all men, but their degrees of membership depend on their height.

Name	Height, cm	Degree of Membership	
		Crisp	*Fuzzy*
Chris	208	1	1.00
Mark	205	1	1.00
John	198	1	0.98
Tom	181	1	0.82
David	179	0	0.78
Mike	172	0	0.24
Bob	167	0	0.15
Steven	158	0	0.06
Bill	155	0	0.01
Peter	152	0	0.00

Crisp and fuzzy sets of “*tall men*”



- The x-axis represents the universe of discourse–the range of all possible values applicable to the variable of choice. In our case, the variable is the man height. According to this representation, the universe of men's heights consists of all tall men.
- The y-axis represents the membership value of the fuzzy set. In our case, the fuzzy set of "tall men" maps height values into corresponding membership values.
- In the fuzzy theory, fuzzy set A of universe X is defined by function $\mu_A(x)$ called the membership function of set A .

$\mu_A(x)$: $X \rightarrow [0, 1]$,

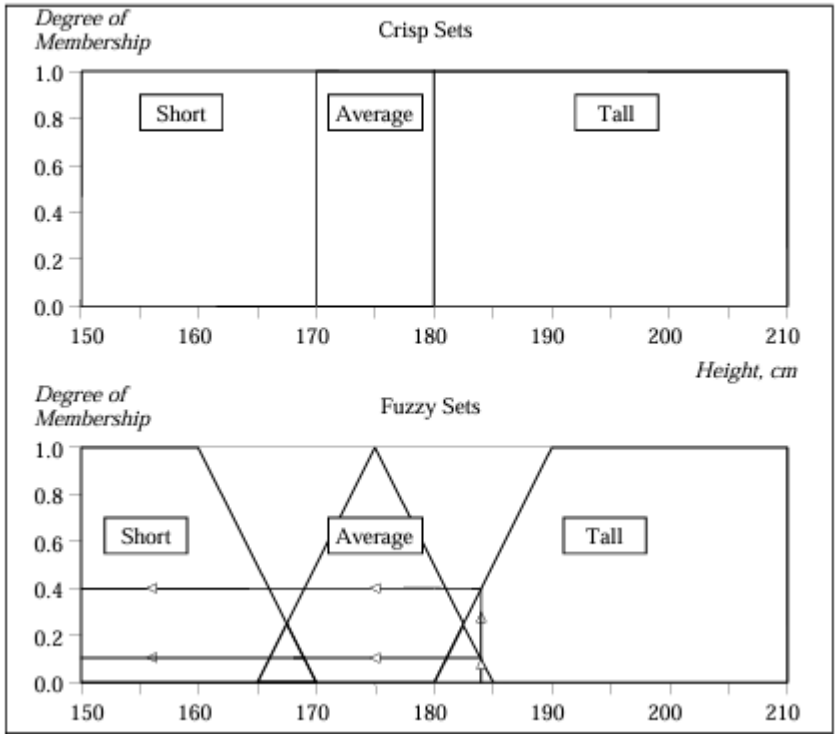
where, $\mu_A(x) = 1$ if x is totally in A;

$\mu_A(x) = 0$ if x is not in A;

$0 < \mu_A(x) < 1$ if x is partly in A.

- This set allows a continuum of possible choices. For any element x of universe X, membership function $\mu_A(x)$ equals the degree to which x is an element of set A. This degree, a value between 0 and 1, represents the degree of membership, also called membership value, of element x in set A.

Crisp and fuzzy sets of short, average, and tall men



- A man who is 184 cm tall is a member of the average men set with a degree of membership of 0.1, and at the same time, he is also a member of the tall men set with a degree of 0.4.

Fuzzy Set Operations:

- **Fuzzy union(∪):** the union of two fuzzy sets is the maximum (MAX) of each element from two sets. [$\mu_C(x) = \mu_{A\cup B} = MAX(\mu_A(x), \mu_B(x)) = \mu_A(x) \vee \mu_B(x)$]

E.g.

$\mu_A(x)$= {1.0, 0.20, 0.75}

$\mu_B(x)$ = {0.2, 0.45, 0.50}

$\mu_C(x)$ = {MAX(1.0, 0.2), MAX(0.20, 0.45), MAX(0.75, 0.50)}= {1.0, 0.45, 0.75}

- **Fuzzy intersection(∩):** the intersection of two fuzzy sets is just the MIN of each element from the two sets. [$\mu_C(x) = \mu_{A\cap B} = MIN(\mu_A(x), \mu_B(x)) = \mu_A(x) \wedge \mu_B(x)$]

E.g.

$\mu_A(x)$ = {1.0, 0.20, 0.75}

$\mu_B(x)$= {0.2, 0.45, 0.50}

$\mu_C(x)$ = {MIN(1.0, 0.2), MIN(0.20, 0.45), MIN(0.75, 0.50)} = {0.2, 0.20, 0.50}

Complement:

- The complement of a fuzzy variable x is (1-x).

- The complement of a fuzzy set is composed of all elements' complement.[$\mu_{\bar{A}}(x) = 1 - \mu_A(x)$]

E.g.

$\mu_A(x) = \{1.0, 0.20, 0.75\}$

$\mu_{\bar{A}}(x) = \{1 - 1.0, 1 - 0.2, 1 - 0.75\} = \{0.0, 0.8, 0.25\}$

Containment or Subset:

- Fuzzy set A is contained in fuzzy set B (or A is a subset of B) if $\mu_A(x) \leq \mu_B(x)$ for all x.

Fuzzy rules:

- The fuzzy set theory is rooted in linguistic variables.
- A linguistic variable is a fuzzy variable. For example, the statement “John is tall” implies that the linguistic variable *John* takes the linguistic value *tall*.
- In fuzzy expert systems, linguistic variables are used in fuzzy rules.
- A fuzzy rule can be defined as a conditional statement in the form:

IF x is A THEN y is B

-where x and y are linguistic variables(Height, Weight, etc); and

-A and B are linguistic values (tall, short, medium, etc).

What is the difference between classical and fuzzy rules?

- A classical IF-THEN rule uses binary logic, for example,

-Rule1:IF speed is > 100 THEN stopping_distance is >50

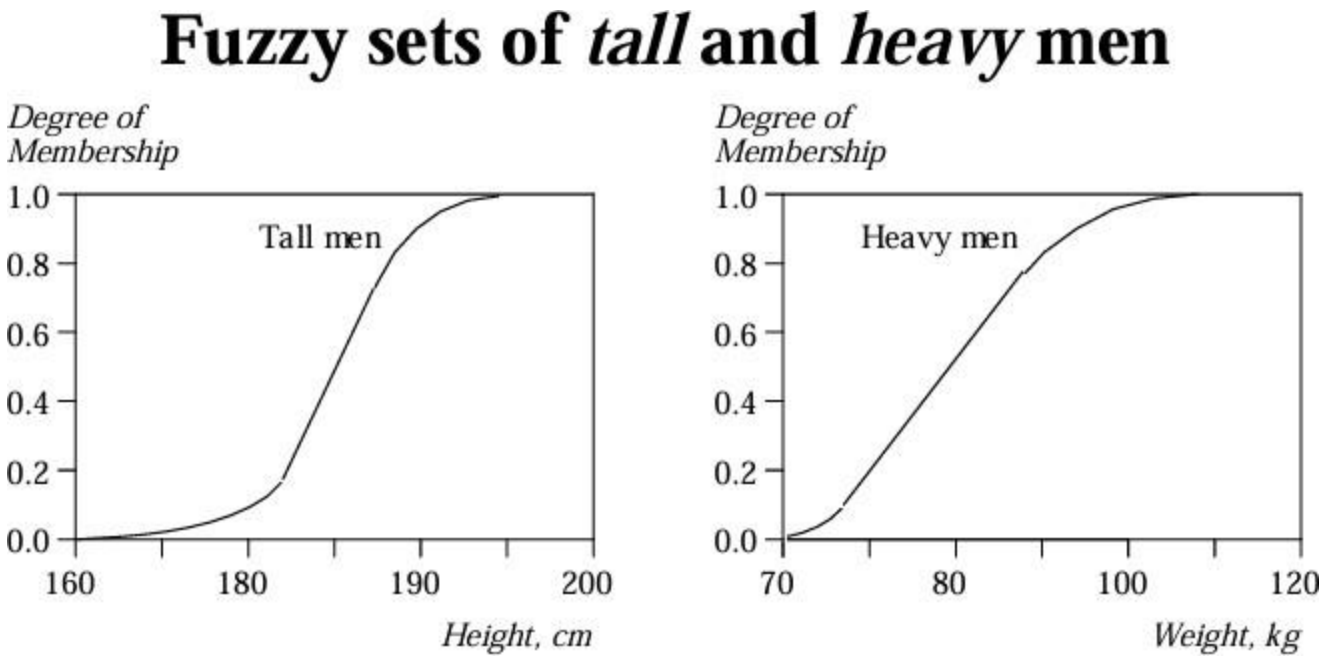
-Rule 2: IF speed is < 40 THEN stopping_distance is 5

- The variable *speed* can have any numerical value between 0 and 220 km/h, but the linguistic variable *stopping_distance* can take either value *long* or *short*.
- If than rules in a fuzzy form:

  -Rule: 1 IF speed is fast THEN stopping_distance is long

  -Rule: 2 IF speed is slow THEN stopping_distance is short
- In fuzzy rules, the linguistic variable *speed* also has the range (the universe of discourse) between 0 and 220 km/h, but this range includes fuzzy sets, such as *slow*, *medium* and *fast*.

- Fuzzy rules relate fuzzy sets.
- In a fuzzy system, all rules fire to some extent, i.e., fire partially. If the antecedent is true to some degree of membership, then the consequent is also true to that same degree.

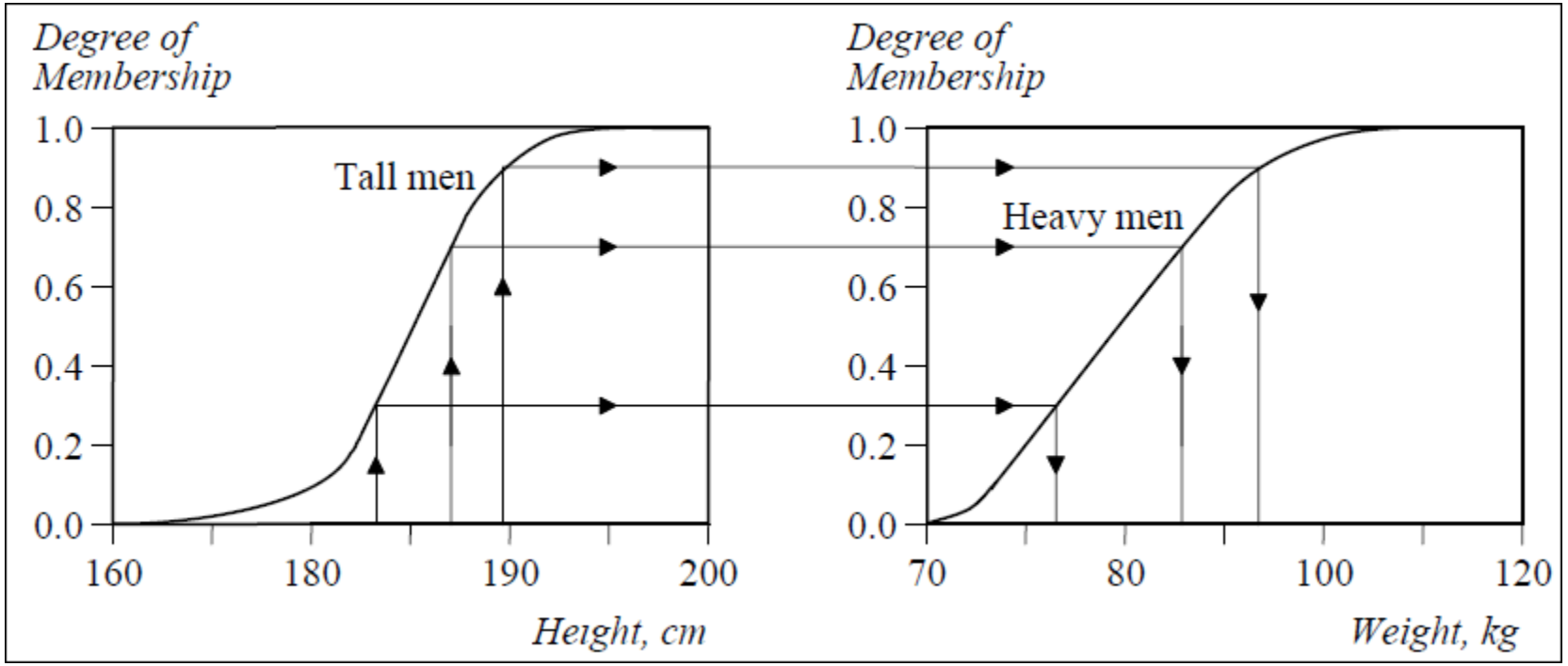


- These fuzzy sets provide the basis for a weight estimation model. The model is based on a relationship between a man's height and his weight:

IF height is *tall*

THEN weight is *heavy*

- The value of the output or a truth membership grade of the rule consequent can be estimated directly from a corresponding truth membership grade in the antecedent. This form of fuzzy inference uses a method called monotonic selection.



- A fuzzy rule can have multiple antecedents, for example:

  IF project_duration is long

  AND project_staffing is large

  AND project_funding is inadequate

  THEN risk is high

  IF service is excellent

  OR food is delicious

  THEN tip is generous

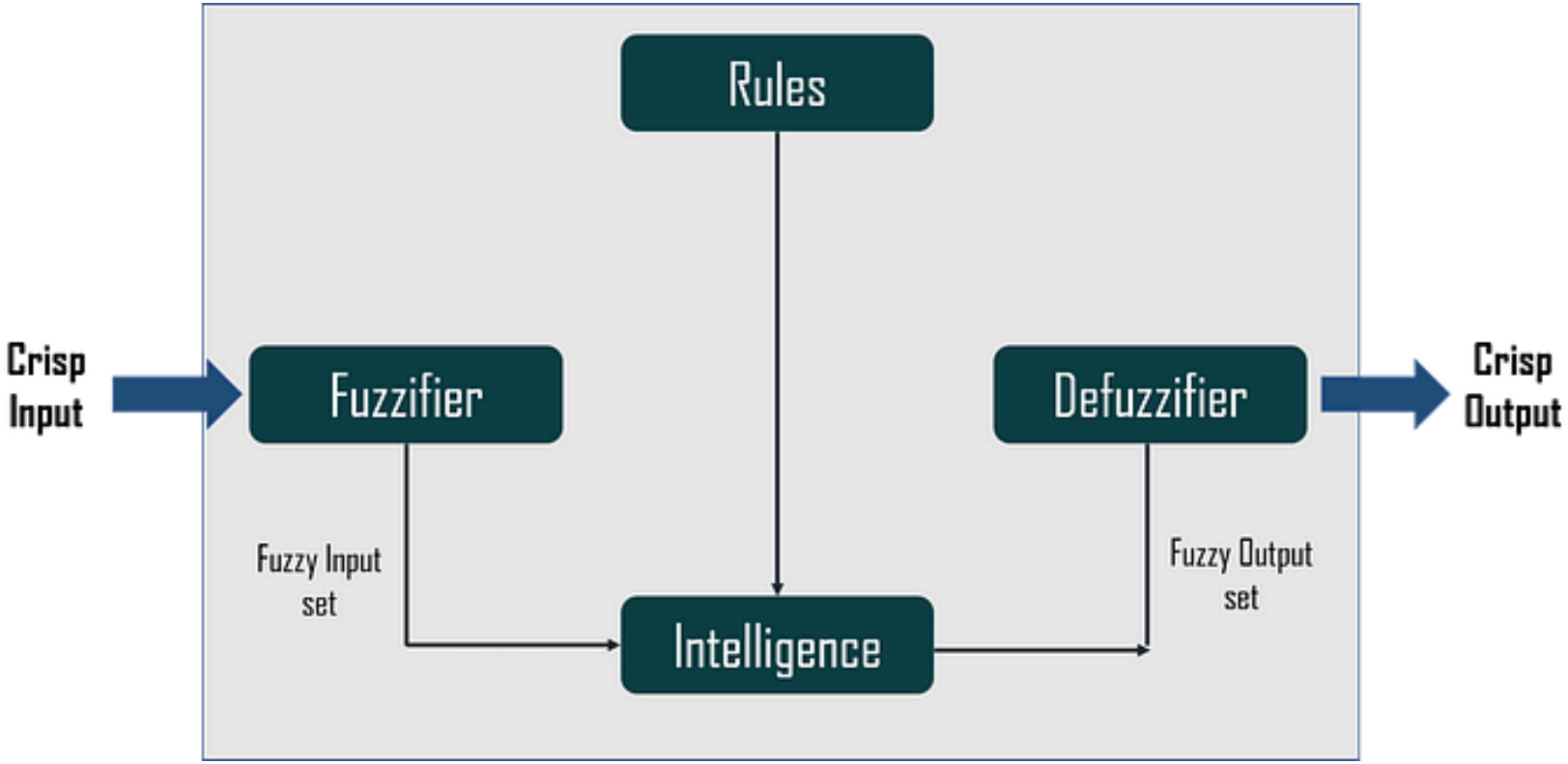
- The consequent of a fuzzy rule can also include multiple parts, for instance:

IF temperature is hot

THEN hot_water is reduced;

cold_water is increased

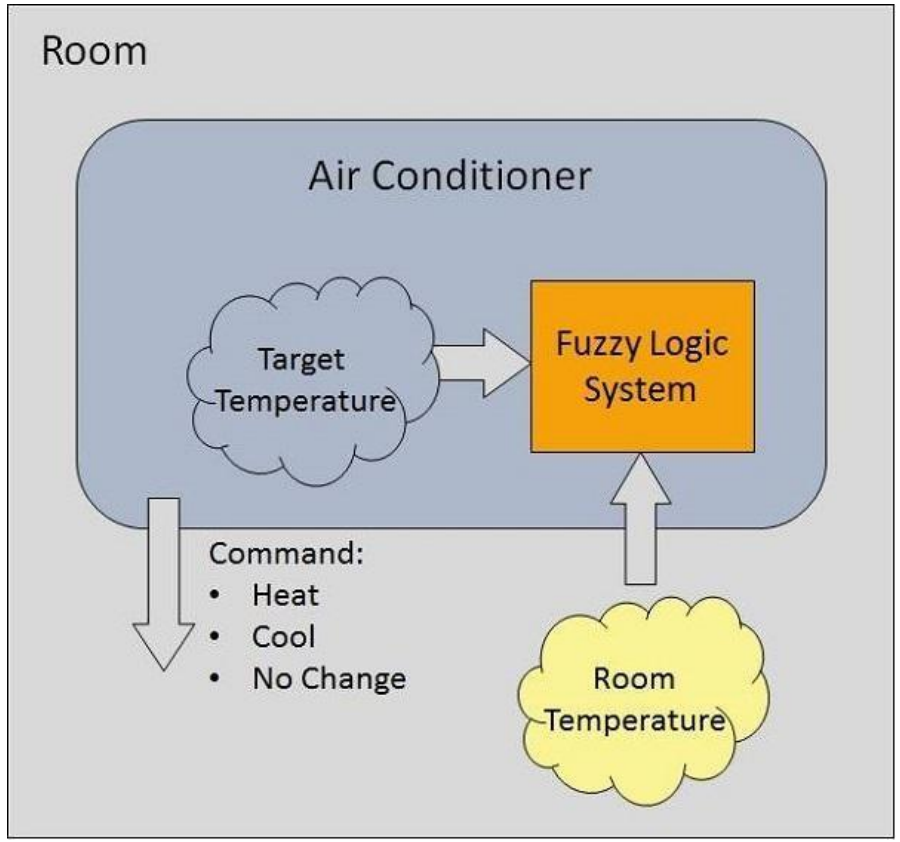
Fuzzy Logic Architecture



- **Fuzzification Module**: It transforms the system inputs, which are crisp numbers, into fuzzy sets. It splits the input signal into five steps such as −**LP,MP,S,MN,LN.**
- **Knowledge Base**: It stores IF-THEN rules provided by experts.
- **Inference Engine**: It simulates the human reasoning process by making fuzzy inference on the inputs and IF-THEN rules.
- **Defuzzification Module**: It transforms the fuzzy set obtained by the inference engine into a crisp value.

Example of a Fuzzy Logic System

Let us consider an air conditioning system with 5-level fuzzy logic system. This system adjusts the temperature of air conditioner by comparing the room temperature and the target temperature value.



Applications of Fuzzy Logic

The Fuzzy logic is used in various fields such as automotive systems, domestic goods, environment control, etc. Some of the common applications are:

- It is used for **decision making support systems** and personal evaluation in the large company business.
- It also controls the pH, drying, chemical distillation process in the **chemical industry**.
- Fuzzy logic is used in **Natural language processing** and various intensive applications in Artificial Intelligence.
- It is extensively used in **modern control systems** such as expert systems.
- Fuzzy Logic mimics how a person would make decisions, only much faster. Thus, you can use it with Neural Networks.

Advantages of FLSs

- Mathematical concepts within fuzzy reasoning are very simple.
- You can modify a FLS by just adding or deleting rules due to flexibility of fuzzy logic.
- Fuzzy logic Systems can take imprecise, distorted, noisy input information.
- FLSs are easy to construct and understand.
- Fuzzy logic is a solution to complex problems in all fields of life, including medicine, as it resembles human reasoning and decision making.

Disadvantages of FLSs

- There is no systematic approach to fuzzy system designing.
- They are understandable only when simple.
- They are suitable for the problems which do not need high accuracy.

Aspect	Fuzzy Logic	Probability
Concept	Deals with **vagueness** and **degrees of truth**	Deals with **uncertainty** and **likelihood of events**
Truth Value	Truth values range between **0 and 1**, representing **partial truth** (e.g., 0.6 true)	Probability values also range from **0 to 1**, but they represent **chance of occurrence**
Focus	Focuses on **how true** a statement is	Focuses on **how likely** an event is to occur
Mathematical Model	Based on **fuzzy set theory**	Based on **probability theory**
Interpretation	"The temperature is warm" may be **partially true** (e.g., 0.7 warm)	"There is a 70% chance of rain tomorrow" expresses **likelihood**
Application	Used in **control systems**, **washing machines**, **AI reasoning**, etc., where imprecision is acceptable	Used in **statistics**, **forecasting**, **risk analysis**, and **machine learning**, where randomness is modeled
Ignorance vs Vagueness	Handles **vagueness** (e.g., how tall is "tall"?)	Handles **ignorance or randomness** (e.g., will it rain or not?)
Logic Nature	Extension of classical logic; allows **graded memberships**	Rooted in **frequentist or Bayesian** interpretation
Example	A room is "somewhat clean" (clean = 0.6)	There's a 60% chance that a die shows a 6
Truth vs Belief	Deals with **degrees of truth**	Deals with **degrees of belief** or **likelihood**